

# Verilogの基礎その3

ISHI会

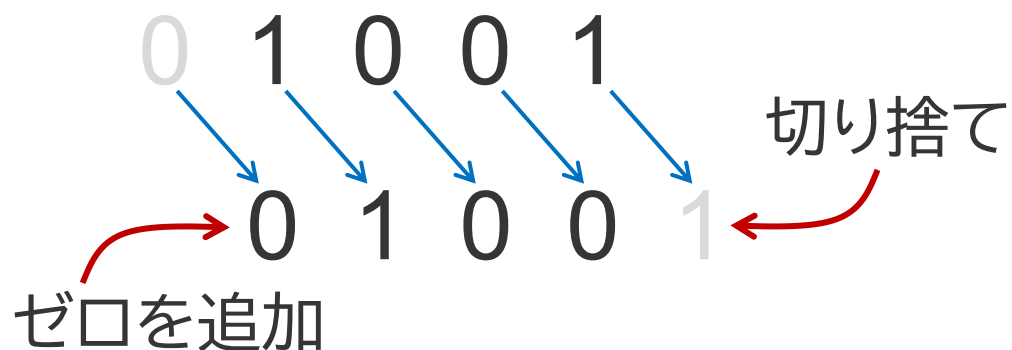
フルデジタルで音声を操るためのロジック回路 3回目

# 目次

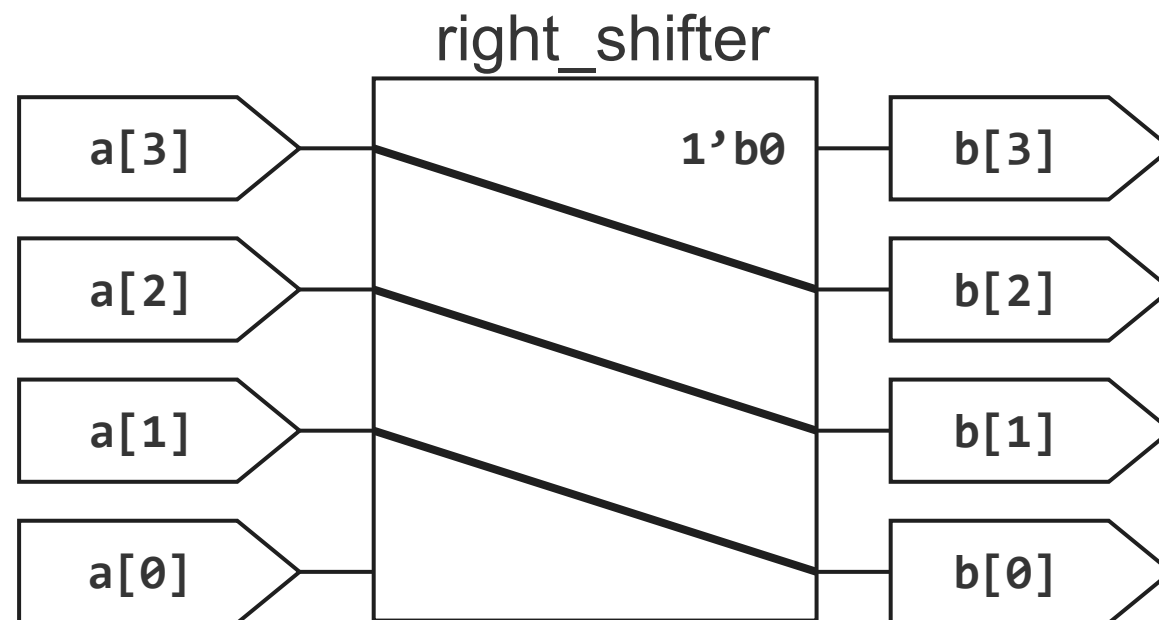
- 先週の回答
- alwaysによる順序回路の記述
- 4ビットカウンター
- クロック分周
- チャタリング除去
- 線形帰還シフトレジスタ
- ステートマシン
- 練習問題

# 右シフター（バスを用いた回路）

各ビットを下記のように順序を維持したまま右側にシフトする回路を右シフターと呼ぶ。このとき、最上位ビットはゼロを付け加え、右にはみ出す最下位ビットは切り捨てられる。



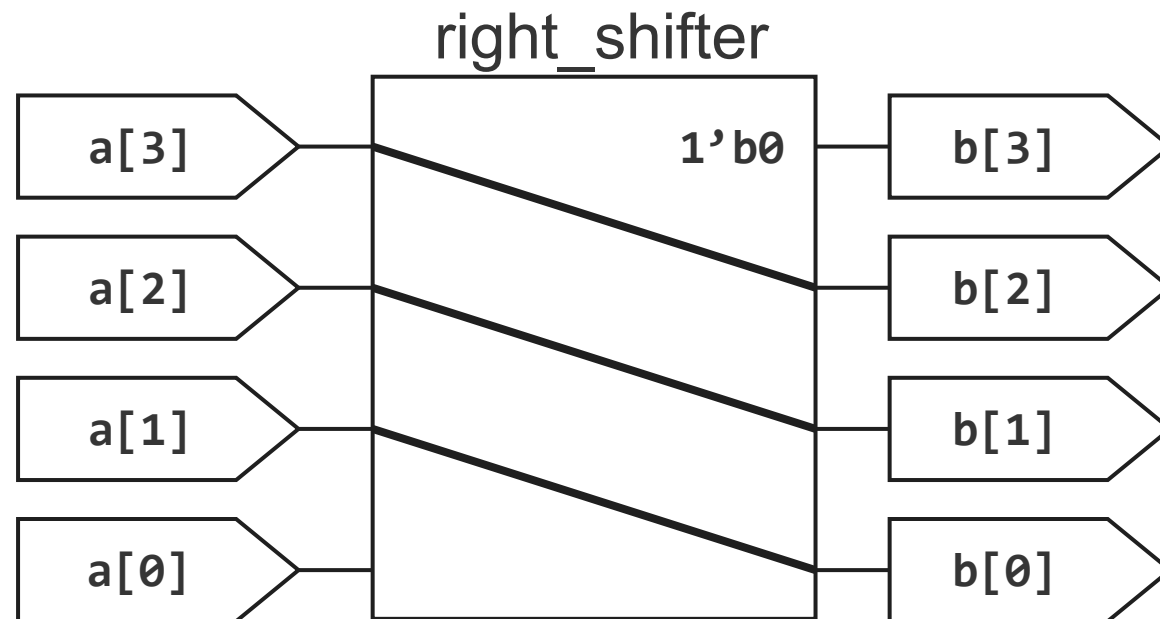
右シフターの回路は右図のように実現することができる。  
入力の最下位ビットは切り捨て、  
出力の最上位ビットにゼロを追加する。



# 右シフターの記述

右の回路をVerilogで記述する

```
module right_shifter(  
    input wire [3:0] a,  
    output wire [3:0] b);  
  
    assign b = {1'b0, a[3:1]};  
endmodule
```



# Half Adder 1

1ビットの2つの信号の和を求める回路をHalf Adder(半加算器)という。

## 1ビット信号の足し算

$$\begin{array}{r} 0 \\ +) 0 \\ \hline 00 \end{array}$$

$$\begin{array}{r} 1 \\ +) 0 \\ \hline 01 \end{array}$$

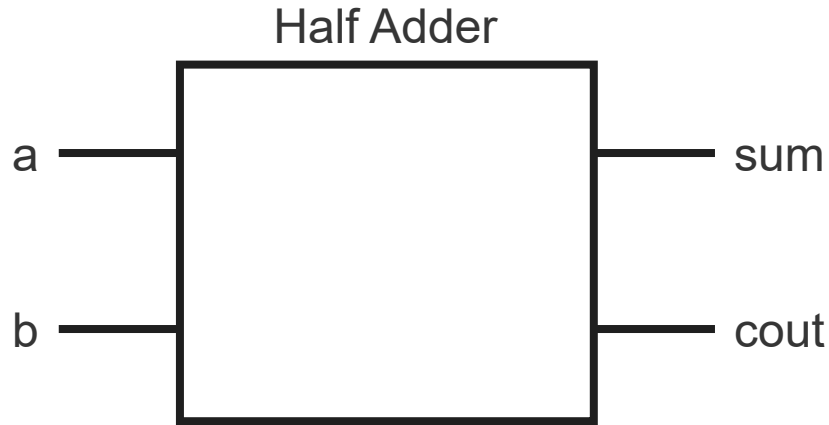
$$\begin{array}{r} 0 \\ +) 1 \\ \hline 01 \end{array}$$

$$\begin{array}{r} 1 \\ +) 1 \\ \hline 10 \end{array}$$



繰り上がりの桁が必要になるので1ビット同士の足し算の出力は2ビットになる。  
繰り上がりの桁はキャリー(Carry)と呼ばれる

# Half Adder 2



| 真理値表 |   |     |      |
|------|---|-----|------|
| a    | b | sum | cout |
| 0    | 0 | 0   | 0    |
| 0    | 1 | 1   | 0    |
| 1    | 0 | 1   | 0    |
| 1    | 1 | 0   | 1    |

↑ 排他的論理和      ↑ 論理積

- Half Adderは2つの入力、2つの出力がある。
- 真理値表の関係を2つの出力それぞれにVerilogコードを書く。

## 2. Half Adder のVerilogコード

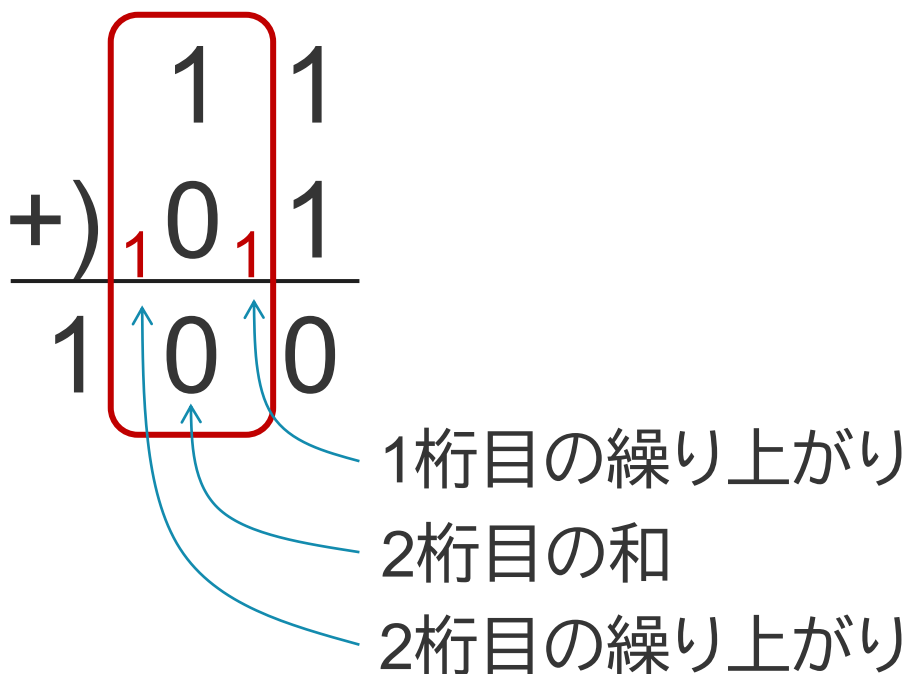
2つの信号aとbの論理和と論理積を求め同時に出力する回路を作成する。

```
module half_adder(  
    input wire a,  
    input wire b,  
    output wire sum,  
    output wire cout);  
  
    assign c = a & b;  
    assign d = a | b;  
endmodule
```

# 全加算器

全加算器は加算器の最小要素を抜き出した回路である。

加算器の最小要素を考えるために2桁(2ビット)の足し算を考える。



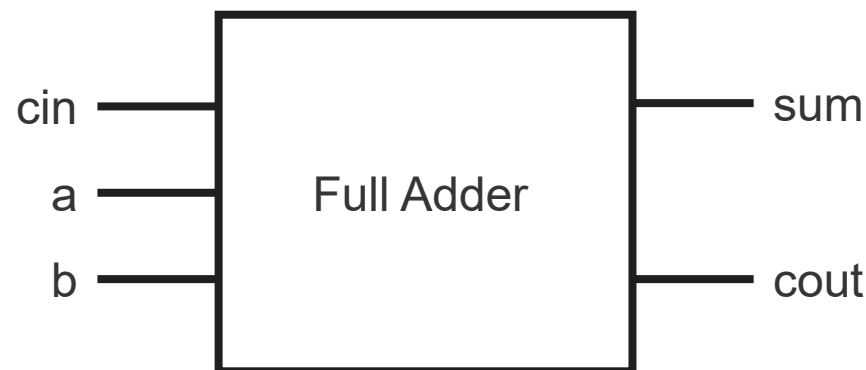
2桁目で加算される数値は、1桁目の繰り上がり、2桁目同士の値の3つである。

したがって、入力3つ必要になる。

和は2桁となり、2桁目の値と繰り上がりの値になる。

したがって、出力は2つとなる。

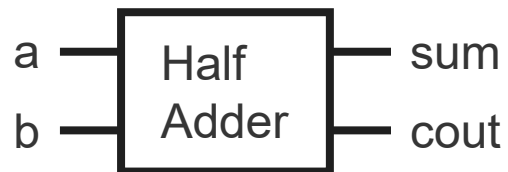
この回路を全加算器と呼び、加算回路を一般化した最小構成となる



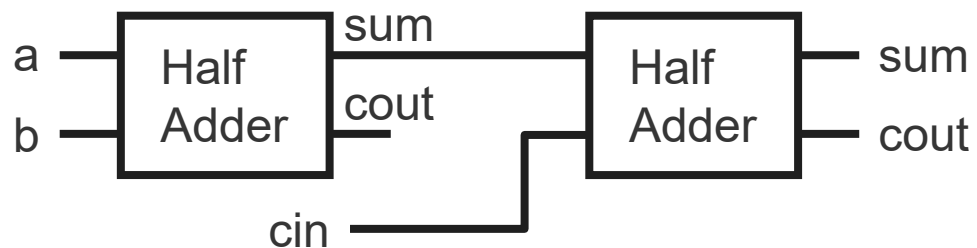


# Full Adderの実現方法

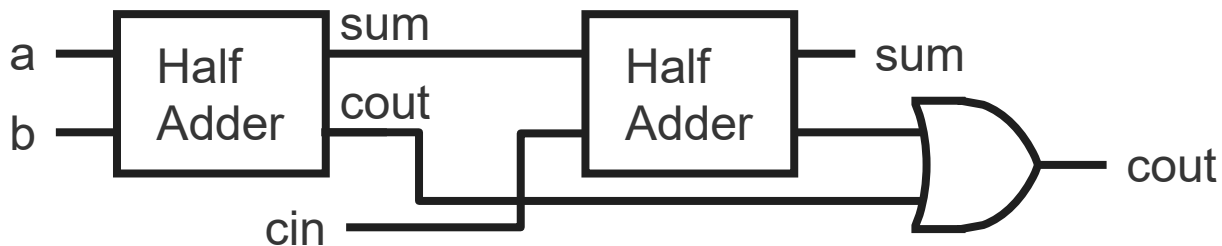
まずaとbの和を考える。これはHalf Adderにより実現される。



次に繰り上がりのcinを加算する。これはHalf Adderを使う。

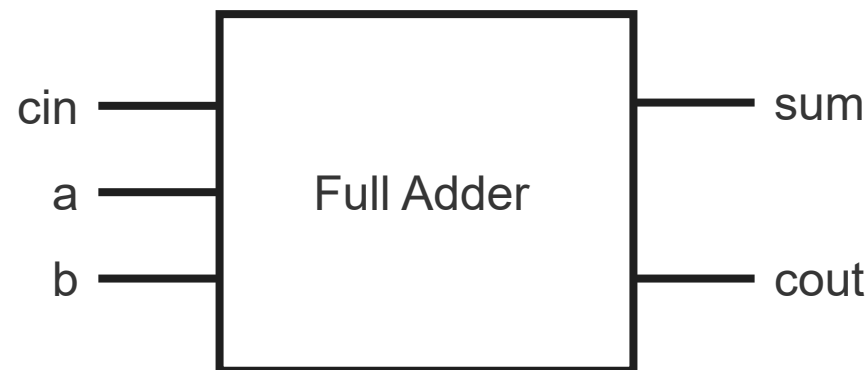


最後にcoutを考える。Full Adderのcoutが1になるのは、それぞれの半加算器のcoutのどちらか(どちらも)が1になるときである。したがって、2つのcoutの論理和を取れば良い。



全加算器の真理値表

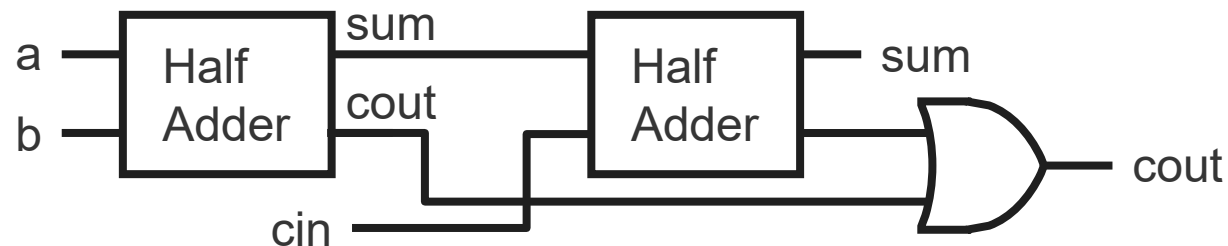
| a | b | cin | sum | cout |
|---|---|-----|-----|------|
| 0 | 0 | 0   | 0   | 0    |
| 0 | 0 | 1   | 1   | 0    |
| 0 | 1 | 0   | 1   | 0    |
| 0 | 1 | 1   | 0   | 1    |
| 1 | 0 | 0   | 1   | 0    |
| 1 | 0 | 1   | 0   | 1    |
| 1 | 1 | 0   | 0   | 1    |
| 1 | 1 | 1   | 1   | 1    |



# Full Adderの実装

```
module full_adder(  
    input wire a,  
    input wire b,  
    input wire cin,  
    output wire sum,  
    output wire cout);  
  
    wire first_sum;  
    wire first_cout;  
    wire second_cout;  
  
    half_adder first(  
        .a(a),  
        .b(b),  
        .sum(first_sum),  
        .cout(first_cout));  
  
    half_adder second(  
        .a(first_sum),  
        .b(cin),  
        .sum(sum),  
        .cout(second_cout));  
  
    assign cout = first_cout | second_cout;  
endmodule
```

Half adderを用いてFull Adderを実装する。



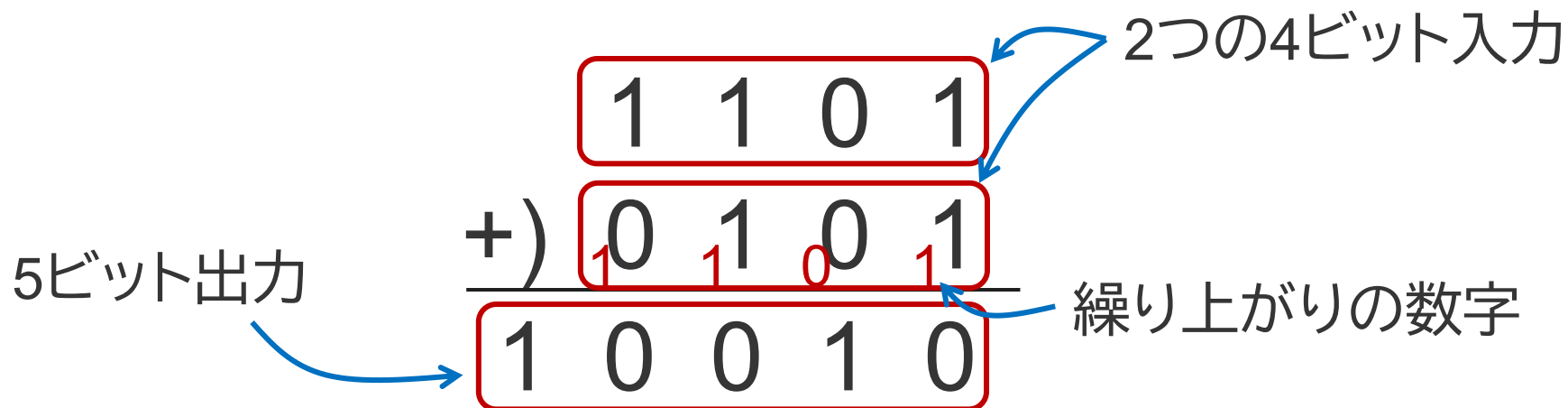
Half adderは1つ前の練習で作成したモジュールを使用する。ファイル名はhalf\_adder.svとする。

Compile Optionsには「half\_adder.sv」を追加する。

# 4ビット加算回路の実装

Full Adderを用いて加算回路を実装する。

2進数で4桁の足し算は筆算から次のように考える。

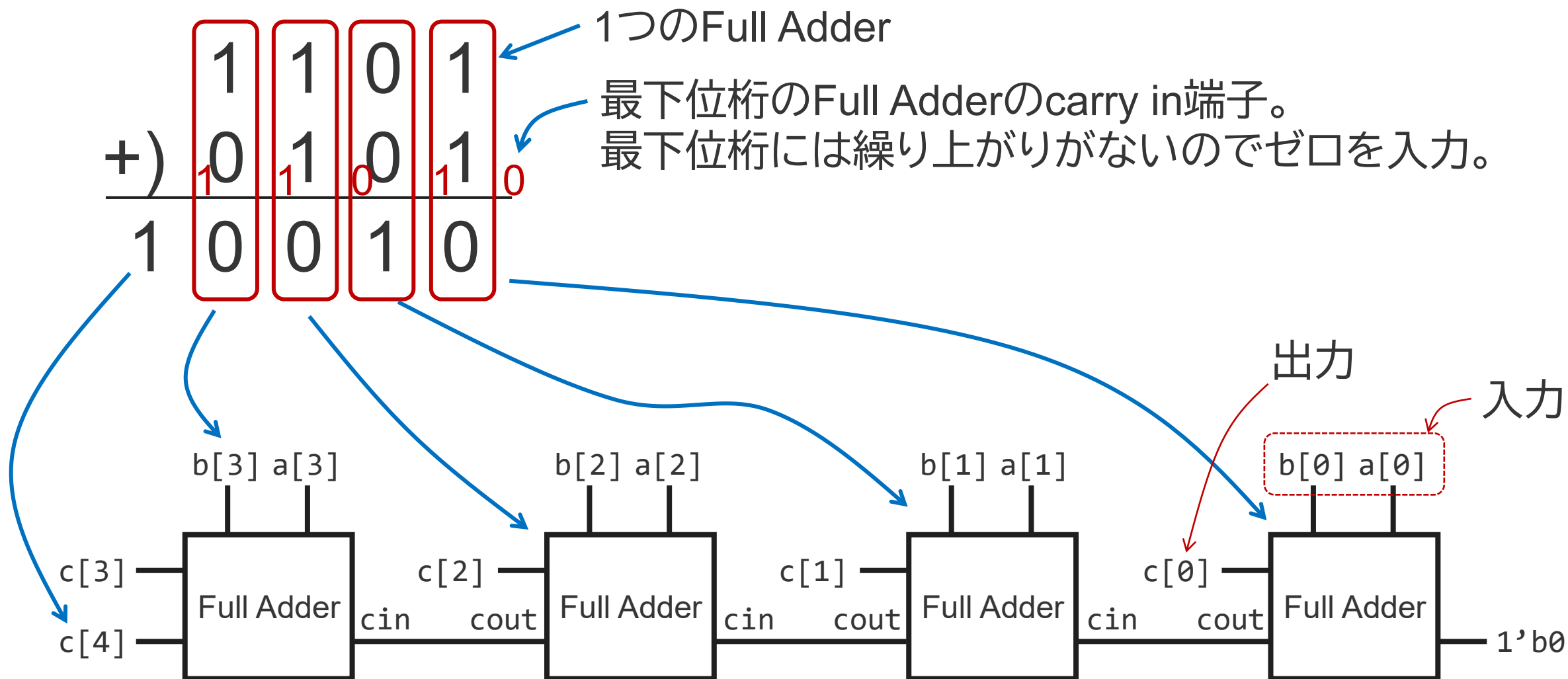


Nビット同士の足し算は繰り上がりからN+1ビットが必要になる。

このことから、4ビット加算回路では入力は4ビットの信号が2つ、出力は4ビットの信号が1つとなる。

# Full Adderを用いた4ビット加算回路

多ビットの加算回路を実現するには各桁を加算回路によって実装する。



# Full Adderを用いた4ビット加算回路の記述

```
module four_bit_adder(  
    input wire [3:0] a,  
    input wire [3:0] b,  
    output wire [4:0] c);  
  
    wire first_cout;  
    wire second_cout;  
    wire third_cout;  
  
    full_adder first(  
        .a(a[0]),  
        .b(b[0]),  
        .cin(1'b0),  
        .sum(c[0]),  
        .cout(first_cout));  
  
    full_adder second(  
        .a(a[1]),  
        .b(b[1]),  
        .cin(first_cout),  
        .sum(c[1]),  
        .cout(second_cout));
```

続<

続き

```
        full_adder third(  
            .a(a[2]),  
            .b(b[2]),  
            .cin(second_cout),  
            .sum(c[2]),  
            .cout(third_cout));  
  
        full_adder fourth(  
            .a(a[3]),  
            .b(b[3]),  
            .cin(third_cout),  
            .sum(c[3]),  
            .cout(c[4]));  
  
    endmodule
```

full\_adder 及び half\_adderモジュールは実習内で作成したものを  
使用する。それぞれ、full\_adder.sv、half\_adder.svとして追加する。  
Compile Optionsには「half\_adder.sv full\_adder.sv」を追加する。

# 2ビット乗算回路1

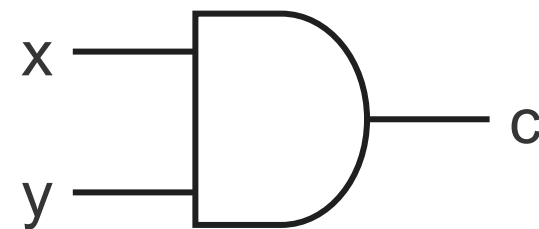
2進数であっても乗算の手順は10進数と同様である。筆算は下記のように計算する。

$$\begin{array}{r}
 11 \\
 \times 11 \\
 \hline
 11 \\
 111 \\
 \hline
 1001
 \end{array}$$

これを回路として実現する方法を考える。回路も筆算の計算順序と同様に実現していく。はじめに1ビット同士の乗算を考える。

1ビット同士の乗算

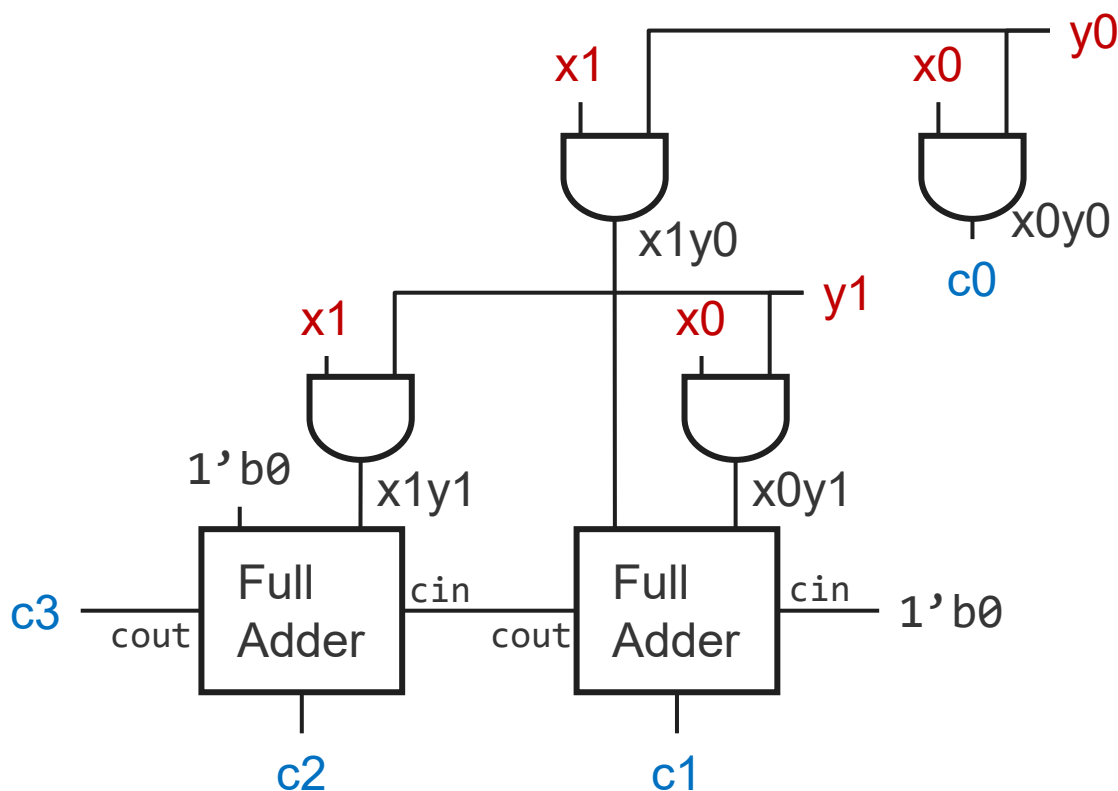
| x | y | c |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |



1ビット同士の乗算の真理値表は論理積と同じであり、1ビット同士の乗算は論理積として表現できる。したがって、筆算の各値は論理積として考えることができる。

## 2ビット乗算回路2

各桁の論理積を並べ、加算する桁にFull Adderをつなぐと下記のようなになる。  
1つ目のFull Adderはキャリー入力にゼロをつなぎ、2つ目のFull Adderは何も繋がらない入力にはゼロを接続する。



入力は赤、出力は青で表記

$$\begin{array}{r}
 \times) \quad x_1 \quad x_0 \\
 \quad y_1 \quad y_0 \\
 \hline
 \quad x_1y_0 \quad x_0y_0 \\
 x_1y_1 \quad x_0y_1 \\
 \hline
 c_3 \quad c_2 \quad c_1 \quad c_0
 \end{array}$$

# 2ビット乗算回路のVerilog記述

```
module two_bit_multiply(  
    input wire [1:0] a,  
    input wire [1:0] b,  
    output wire [3:0] c);
```

```
    wire x0y0;  
    wire x0y1;  
    wire x1y0;  
    wire x1y1;
```

```
    wire first_cout;
```

```
    assign x0y0 = a[0] & b[0];  
    assign x0y1 = a[0] & b[1];  
    assign x1y0 = a[1] & b[0];  
    assign x1y1 = a[1] & b[1];
```

続<

続き

```
        full_adder first(  
            .a(x0y1),  
            .b(x1y0),  
            .cin(1'b0),  
            .sum(c[1]),  
            .cout(first_cout));
```

```
        full_adder second(  
            .a(x1y1),  
            .b(1'b0),  
            .cin(first_cout),  
            .sum(c[2]),  
            .cout(c[3]));
```

```
        assign c[0] = x0y0;
```

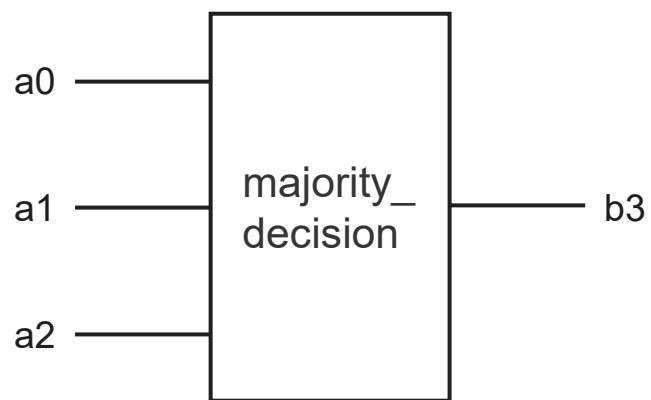
```
endmodule
```

full\_adder 及び half\_adderモジュールは実習内で作成したものを使用する。それぞれ、full\_adder.sv、half\_adder.svとして追加する。Compile Optionsには「half\_adder.sv full\_adder.sv」を追加する。



# 多数決回路

多数決回路は入力3ビットのうち、2ビット以上が1のときに1が出力される回路である。



| a0 | a1 | a2 | b0 |
|----|----|----|----|
| 0  | 0  | 0  | 0  |
| 0  | 0  | 1  | 0  |
| 0  | 1  | 0  | 0  |
| 0  | 1  | 1  | 1  |
| 1  | 0  | 0  | 0  |
| 1  | 0  | 1  | 1  |
| 1  | 1  | 0  | 1  |
| 1  | 1  | 1  | 1  |

# 多数決回路のVerilog記述

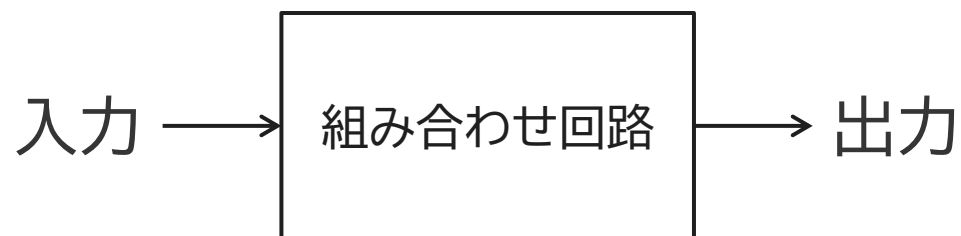
```
module majority_decision(  
    input wire [2:0] a,  
    output reg b);  
  
    always @(*)  
    begin  
        case(a)  
            3'b000: b = 1'b0;  
            3'b001: b = 1'b0;  
            3'b010: b = 1'b0;  
            3'b011: b = 1'b1;  
            3'b100: b = 1'b0;  
            3'b101: b = 1'b1;  
            3'b110: b = 1'b1;  
            3'b111: b = 1'b1;  
            default: b = 1'b0;  
        endcase  
    end  
endmodule
```

# always文を用いた 順序回路の記述

# 組み合わせ回路と順序回路

## 組み合わせ回路

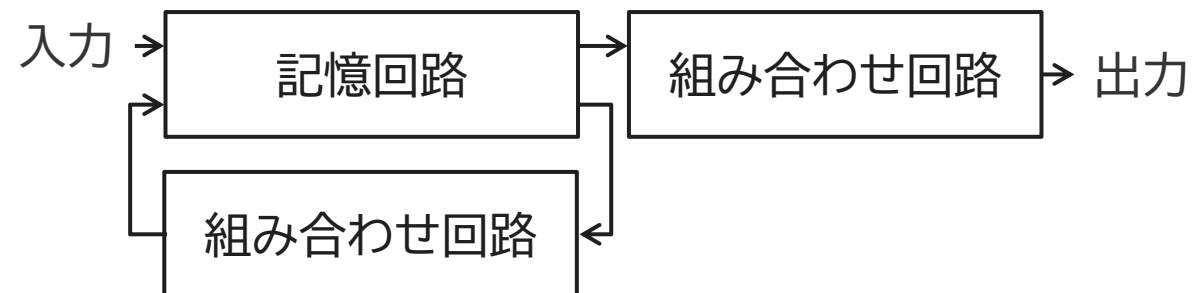
入力に対して出力が一意に決まる回路



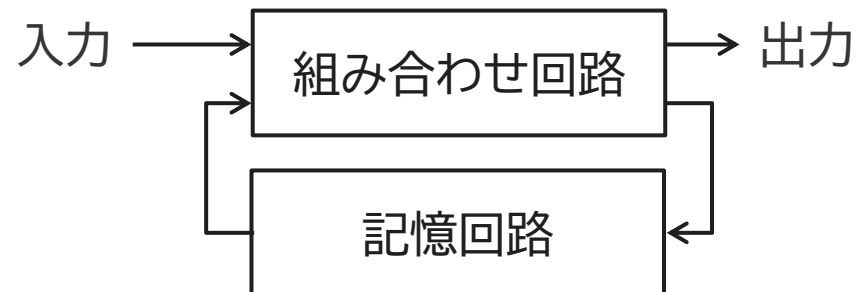
## 順序回路

出力が現在の入力と過去の入力から決まる回路

### ムーアマシン



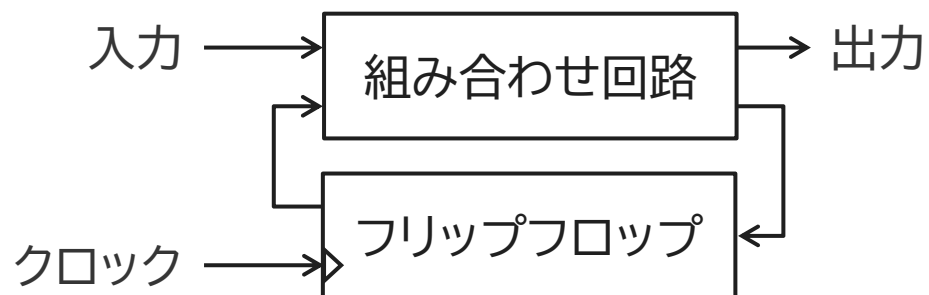
### ミリアマシン



# 同期式順序回路と非同期式順序回路

## 同期式順序回路

クロックパルスごとに記憶素子を更新する回路



### メリット

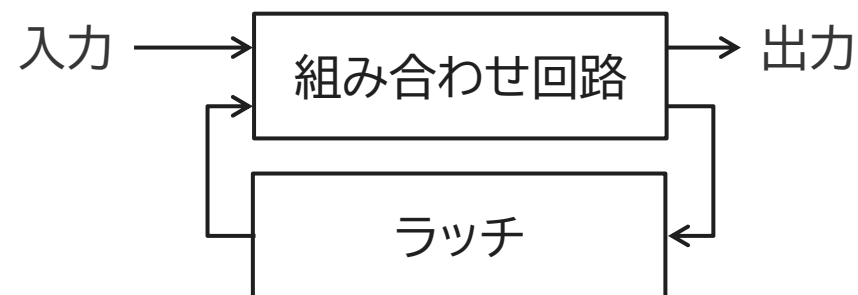
組み合わせ回路の変化が収束した定常状態のみ扱うので設計が簡単

### デメリット

クロックパルスの周期より変化の早い信号は扱えない

## 非同期式順序回路

入力の信号変化で記憶素子を更新する回路



### メリット

処理速度は素子の動作限界で決まるので高速になりやすい

### デメリット

組み合わせ回路の過渡的な状態で誤動作を起こすので設計が困難

FPGAは同期式順序回路を前提とした回路構成

# reg

reg構文を用いることで記憶素子を定義できる。ただし、Verilogにおけるreg構文は合成結果として記憶素子にならないこともあり、記憶素子になるregの使い方を学習する。reg構文は、

```
reg reg_name;
```

のように記述する。regの後ろには、配線名をつける。これで、reg\_nameというレジスタ(配線)ラベルを新たに定義できる。

# always文

always文は信号に変化があったときに実行する動作を書くことができる構文。次のように記述する

```
always @(posedge clk)
```

「always」のあとの「@」から後ろをセンシティビティリストと言う。

フリップフロップはクロックの立ち上がり信号で動作するように記述するので、立ち上がりを意味する「posedge」を信号ラベルの前に付与する。

# ノンブロッキング代入

順序回路でレジスタに値を記憶するときは「<=」を使う

```
always @(posedge clk)
begin
    val <= 4'h3;
end
```

always文では「=」と「<=」の2種類の代入方法がある。  
それぞれ、「ブロッキング代入」と「ノンブロッキング代入」という。

基本的には...

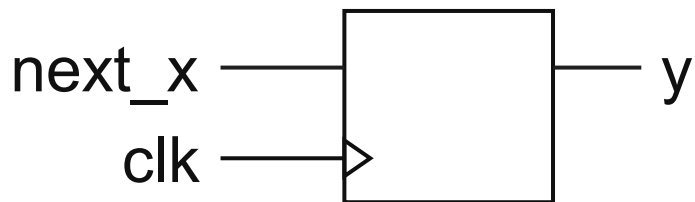
- ✓ ブロッキング代入...組み合わせ回路で使う
  - ✓ ノンブロッキング代入...順序回路で使う
- と覚える。



# ブロッキング代入とノンブロッキング代入の違い

## ブロッキング代入

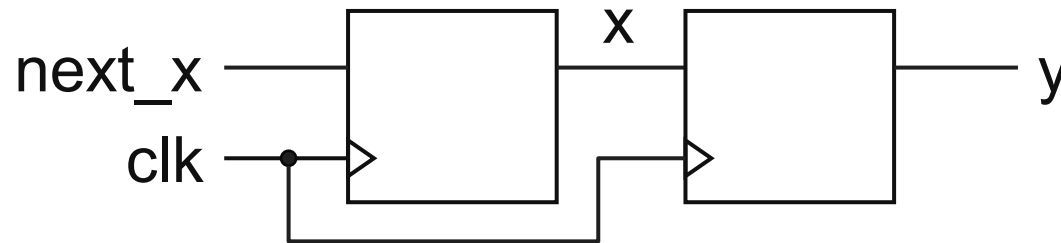
```
always @(posedge clk)
begin
    x = next_x;
    y = x;
end
```



後の代入は前の代入が終わってから実行されるので、1クロック内でこの動作が行われると結果としてxレジスタが消えてしまう。

## ノンブロッキング代入

```
always @(posedge clk)
begin
    x <= next_x;
    y <= x;
end
```

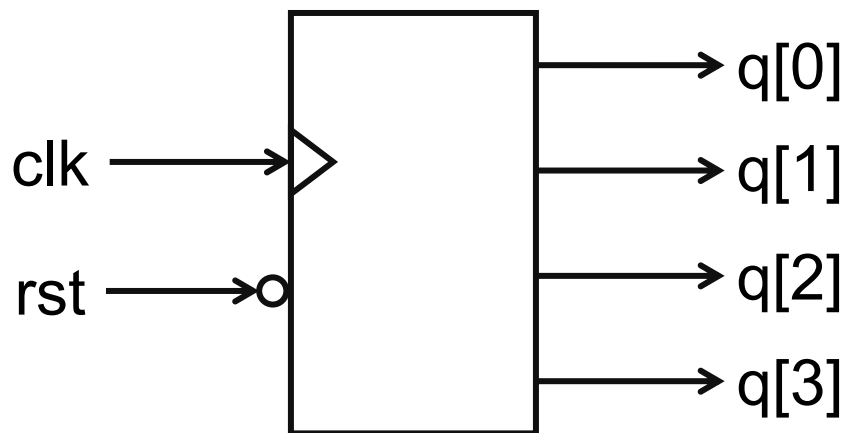


後の代入は前の代入にブロックされないの  
で、xとyそれぞれの代入が並列して行われる。  
結果として2つのレジスタが生成される。

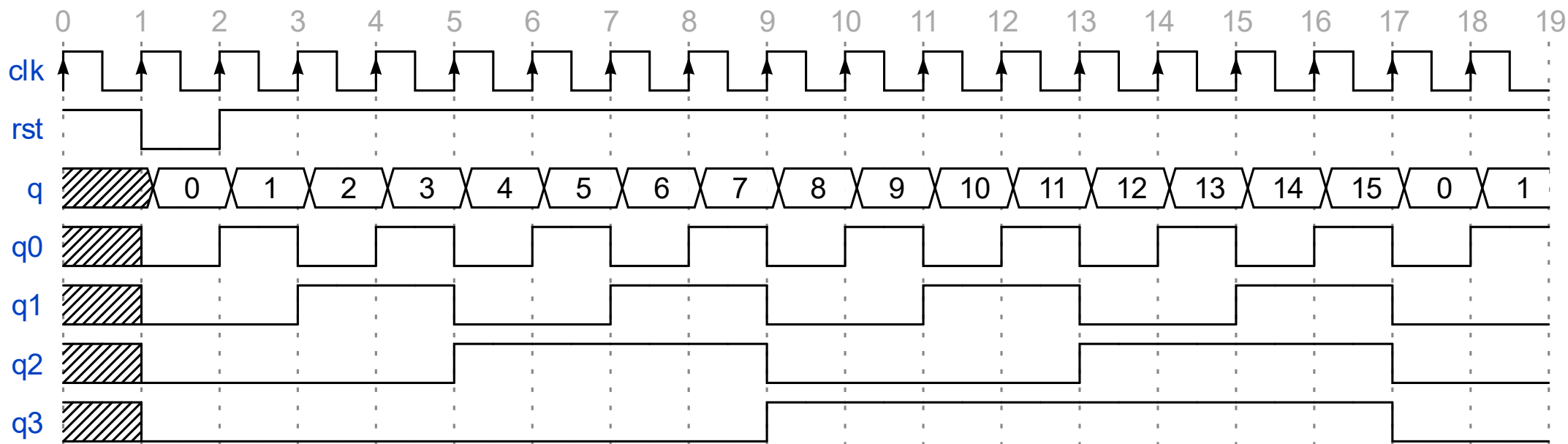
順序回路を期待する記述で消えるレジスタがあるとバグの温床になるので、順序回路ではノンブロッキングのみを使うことが推奨される。

# 4ビットカウンタ

# 4ビットカウンタ回路

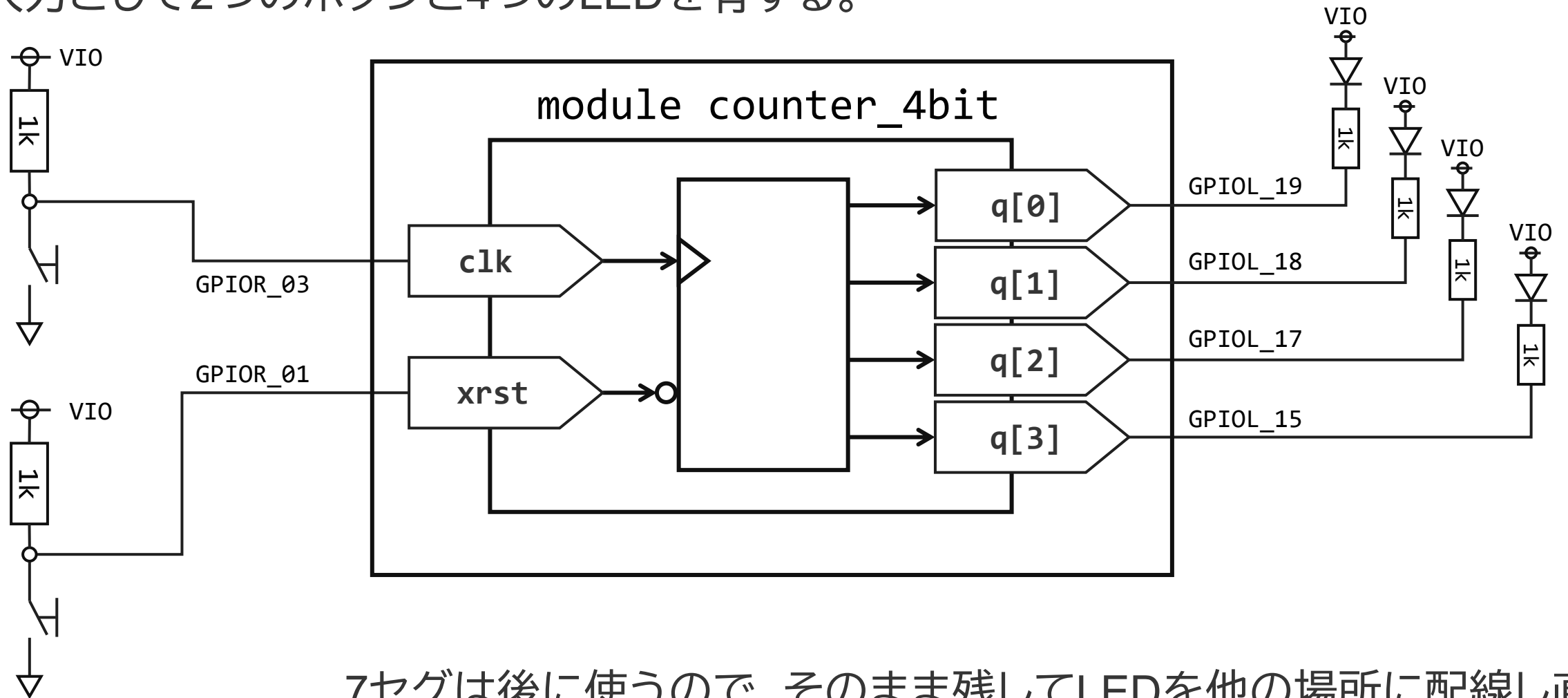


- `clk`入力を15までカウントする4ビットカウンタ
- `rst`は負論理であり、0が入力されたときに出力を0にする。
- `rst`は`clk`に同期



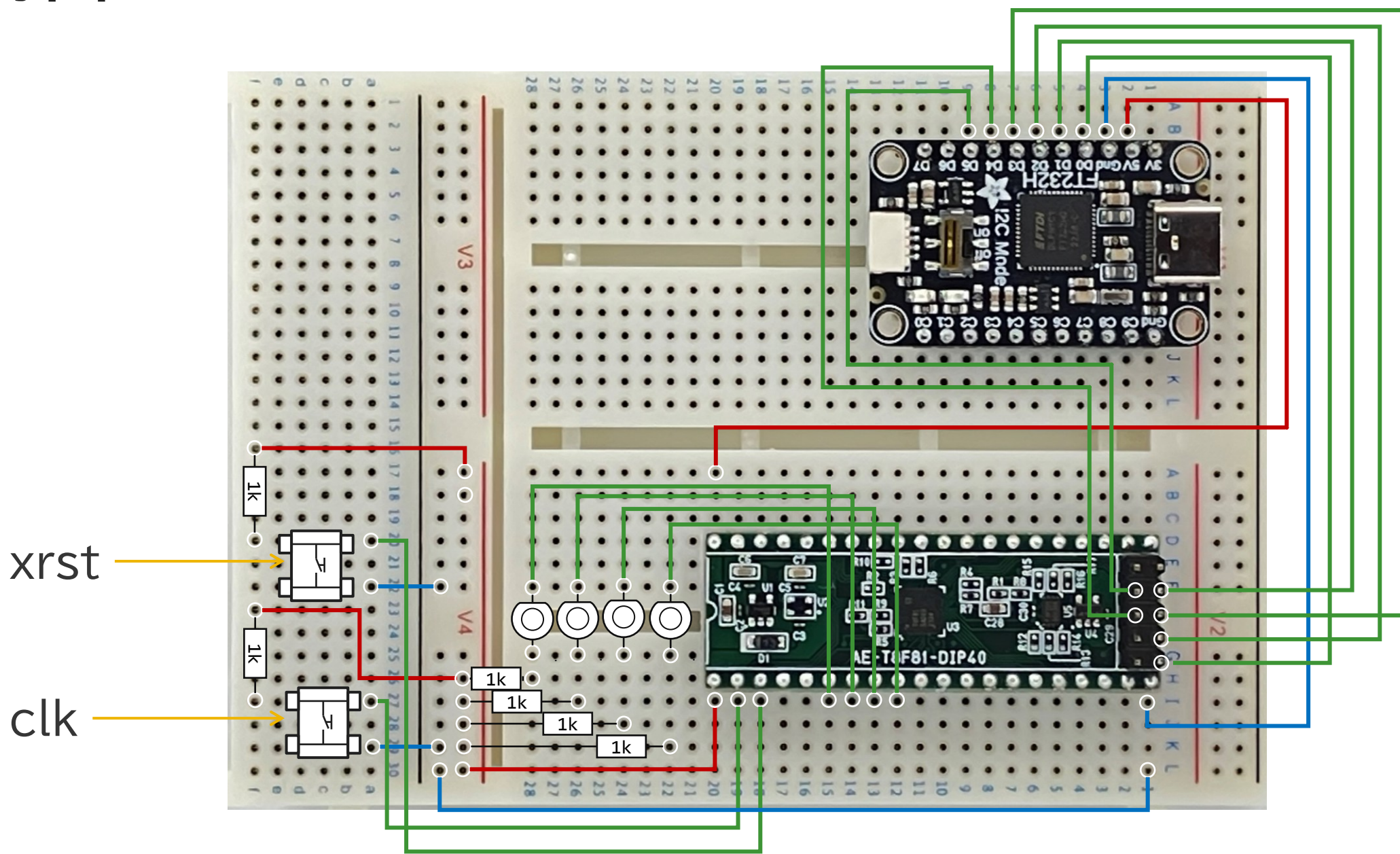
# 4ビットカウンターの確認回路

4ビットカウンタとして動作する回路をFPGAに実装する。  
入力として2つのボタンと4つのLEDを有する。



7セグは後に使うので、そのまま残してLEDを他の場所に配線し直す

# 配線図



# 4ビットカウンタ回路のVerilogコード

モジュール名をcounter\_4bitとしてVerilogコードを入力する。

```
1 module counter_4bit(  
2   input wire clk,  
3   input wire rst,  
4   output reg [3:0] q);  
5  
6   always @(posedge clk)  
7   begin  
8       if (!rst) q <= 4'b0;  
9       else  
10      begin  
11          q <= q + 4'b1;  
12      end  
13  end  
14  
15  endmodule  
16
```

リセット時の動作。リセットが0のときに0を出力

カウンタ回路は複数のフリップフロップをカタマリと捉えて、そのビット列が更新のたびに1ずつ加算される。

Verilogでは一般の四則演算をレジスタに対しても適用可能であり、一般のプログラム言語と同様に+、-、\*、/が使用できる。

積と商は回路規模が大きくなり、動作速度が下がることもあるため、適用は十分に検討する。

Verilogは特に指定しない場合、レジスタ、数字リテラルを符号なし(unsigned)として扱うので注意する。

# ピンアサイン

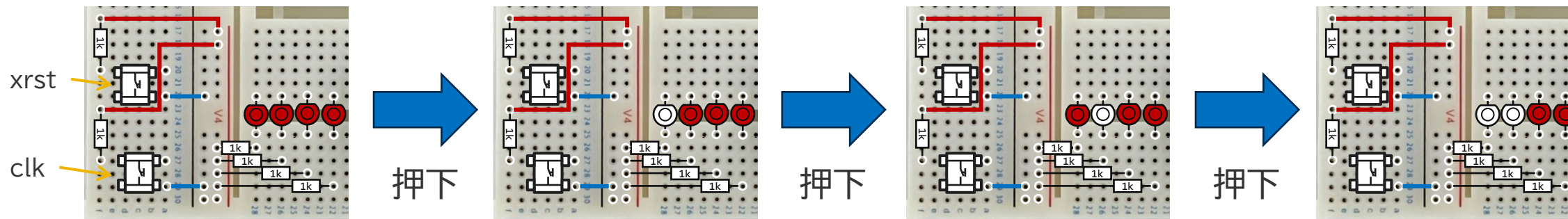
GPIO : Instance View

|            |             |          | all      | all      | all      | all          |                |
|------------|-------------|----------|----------|----------|----------|--------------|----------------|
| Instance ^ | Package Pin | Resource | I/O Bank | Alt Conn | Features | Clock Region | Pad            |
| clk        | A6          | GPIOR_03 | 2A       | None     | None     | R1           | GPIOR_03       |
| q[0]       | D3          | GPIOL_19 | 1B       | GCTRL    | None     | L1           | GPIOL_19_CTRL3 |
| q[1]       | E3          | GPIOL_18 | 1B       | GCTRL    | None     | L1           | GPIOL_18_CTRL2 |
| q[2]       | D2          | GPIOL_17 | 1B       | GCLK     | None     | L1           | GPIOL_17_CLK3  |
| q[3]       | E1          | GPIOL_15 | 1A       | GCLK     | None     | L0           | GPIOL_15_CLK1  |
| xrst       | B5          | GPIOR_01 | 2A       | None     | None     | R1           | GPIOR_01       |



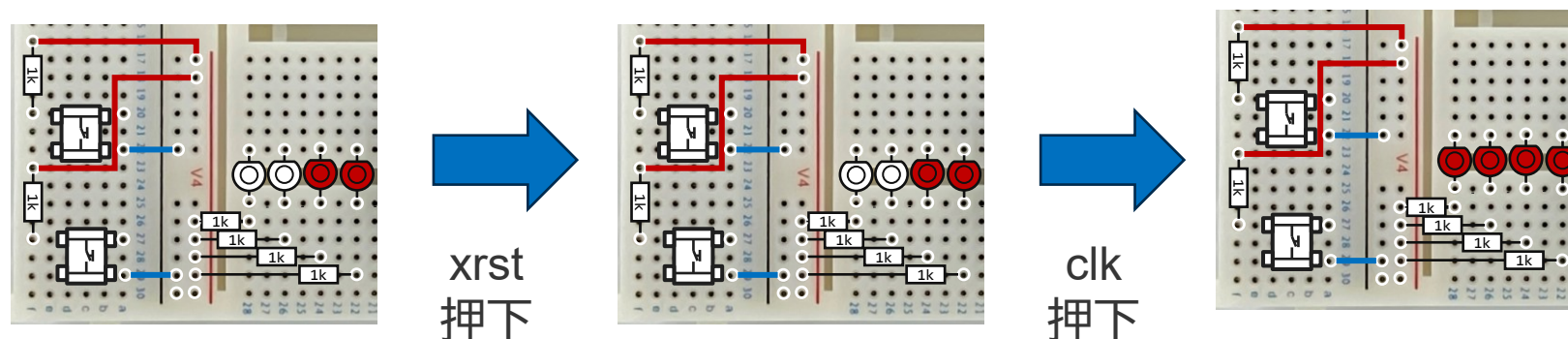
# 動作

- clkを押下



clkの押下に合わせてLEDが2進数のカウントダウンをする

- xrstを押下しながら、clkを押下



xrstを押下した状態でclkを押下することで、LEDは初期状態に戻る

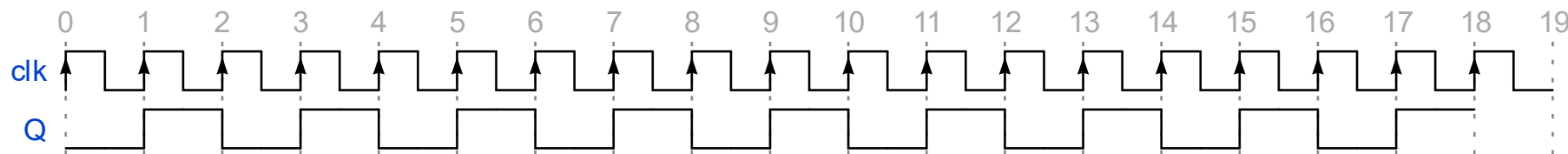
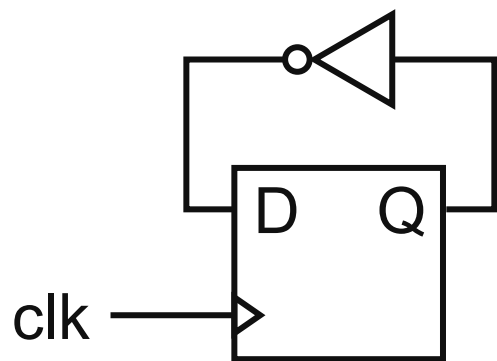
リセットがclkに同期するので、同期リセットと呼ぶ



# クロック分周回路

# クロック分周回路とは？

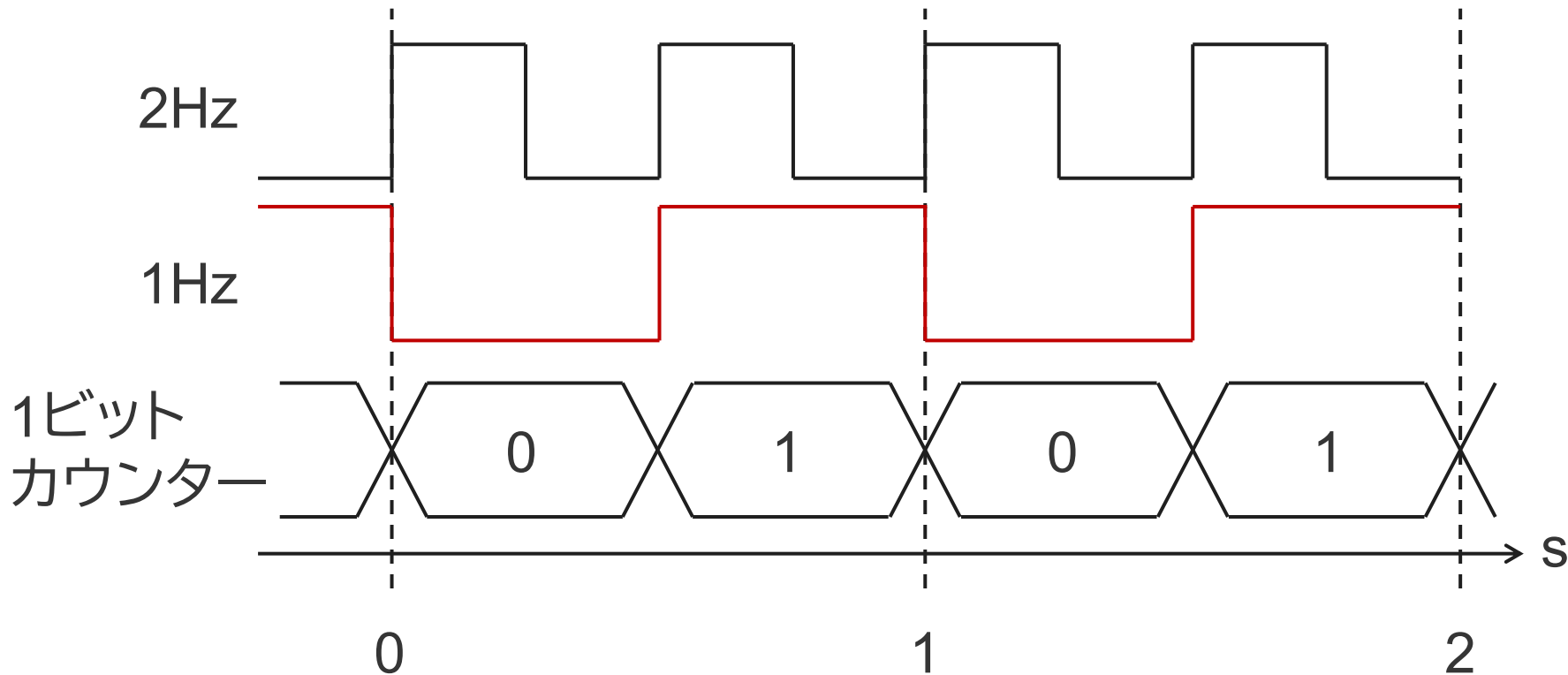
入力したクロック周波数を $n$ 分の1の周波数に変換する回路。  
例えば、D-FF 1個は2分の1の周波数を作り出すことができる。



カウンタを用いることで任意の分周比のクロックを作ることができる。  
ここではオンボードの25MHzクロックから、1Hzのクロックを作成する。

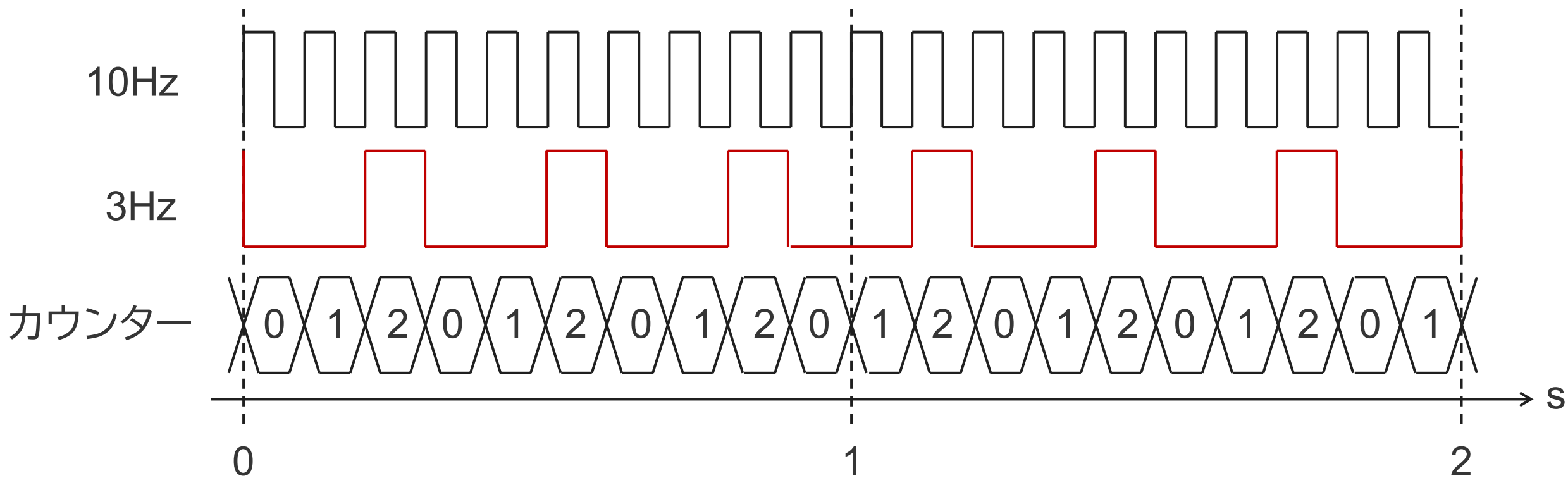
# 分周器の設計方法

2Hzから1Hzを作り出すことを考える。1ビットカウンタを用いてクロックをカウントする。カウンタは0と1を繰り返す。0の期間はLを出力し、1の期間はHを出力する。分周比( $2/1=2$ )の回数だけカウントすると、所望の周波数(1Hz)の1周期になる。したがって、半分のカウントで出力を切り替えることで所望の周波数が得られる。



# 10Hzから3Hzを得るには？

割り切れない分周比を設定することも可能であるが、デューティーが50%でなくなるので注意する。カウントを切り捨てで処理するか、切り上げで処理するかは製作者が選ぶ。(ここでは切り捨てで処理している)



# 25MHzから1Hzを作るVerilogコード

```
1 module clock_div(  
2   input wire f1,  
3   input wire rst,  
4   output reg f2);  
5  
6   localparam MAX_COUNT = 25000000;  
7   reg [25:0] counter;  
8  
9  
10  always @(posedge f1)  
11  begin  
12    if (!rst) counter <= 26'b0;  
13    else  
14    begin  
15      if(counter <= (MAX_COUNT - 1'b1))  
16        counter <= counter + 1'b1;  
17      else  
18        counter <= 26'b0;  
19    end  
20  end  
21  
22  always @(posedge f1)  
23  begin  
24    if (!rst) f2 <= 1'b0;  
25    else  
26    begin  
27      if(counter >= (MAX_COUNT >> 1))  
28        f2 <= 1'b1;  
29      else  
30        f2 <= 1'b0;  
31    end  
32  end  
33  
34 endmodule
```

f2はregで定義する

localparamはモジュール内で使用できる定数

26ビットのカウンターを定義する。

f2はf1に同期して出力させる。

1250万回で出力を切り替える。

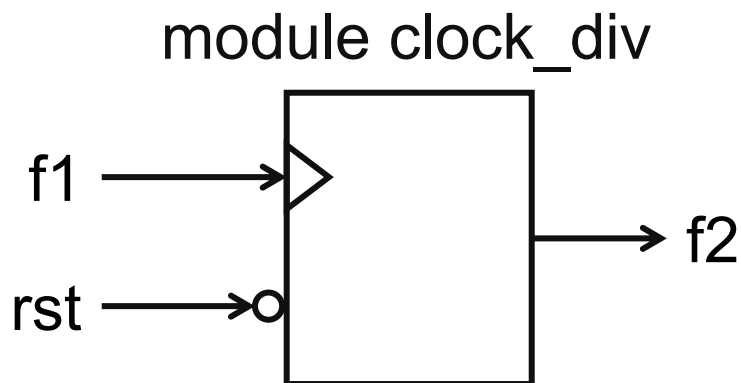
# クロック分周の練習

FPGAボードには25MHzのクロック源が搭載されているので、ここから1Hzのクロックを生成する。

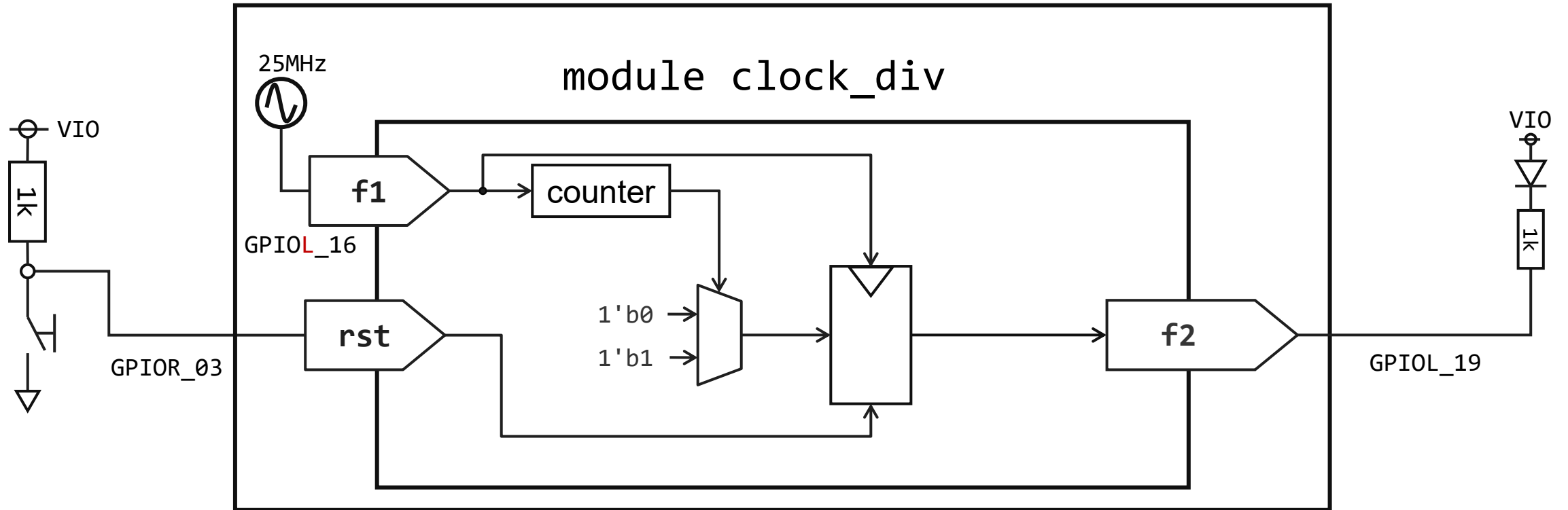
このためには25M回(2500万回)のカウントをできるレジスタを用意する。

$\log_2(25,000,000)$ を計算すると、ざっくり26ビットあると足りる。

ここから1Hzを作るには0~12,500,000までは0を出力、12,500,001~24,999,999は1を出力することで1Hzのクロックが得られる。



# クロック分周の確認回路



# ピンアサイン

40

GPIO : Instance View

|            |             |          | all      | all      | all      | all          |                |
|------------|-------------|----------|----------|----------|----------|--------------|----------------|
| Instance ^ | Package Pin | Resource | I/O Bank | Alt Conn | Features | Clock Region | Pad            |
| ➤ f1       | C2          | GPIOL_16 | 1B       | GCLK     | None     | L1           | GPIOL_16_CLK2  |
| ➤ f2       | D3          | GPIOL_19 | 1B       | GCTRL    | None     | L1           | GPIOL_19_CTRL3 |
| ➤ xrst     | A6          | GPIOR_03 | 2A       | None     | None     | R1           | GPIOR_03       |



# Attention!

FPGAでは分周回路により所望の周波数を得ることは推奨されていません。

一般にFPGAではD-FF出力を他のD-FFのクロック入力に接続する際のタイミング解析はできません。

したがって、分周回路で駆動される回路のタイミング解析は行えなくなるため、動作の保証ができなくなります。

D-FFのクロックにはクロック専用回路が使われますが、この回路のレイテンシはタイミング解析を行うことができます。

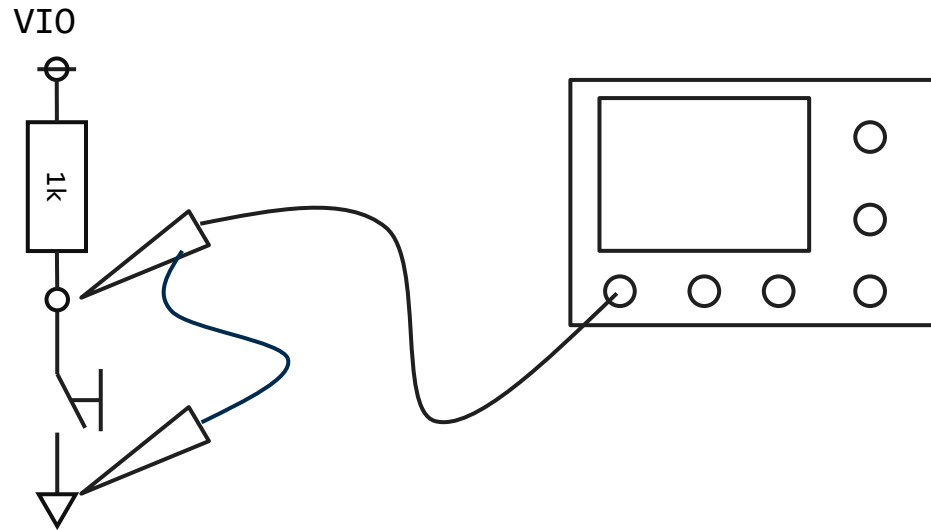
FPGAで複数のクロックを必要とするときはPLLを用いて生成するようにします。

PLLは専用回路のためFPGAの規模に応じて搭載されている個数が異なります。はじめにシステム設計をしっかりと行って必要クロック数を数え上げましょう。それから、適したFPGAを選択するようにします。

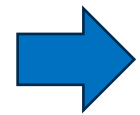
今回はタイミング解析は行っていないませんが、気になる人は調べてやってみましょう。

# チャタリング除去

# スイッチのチャタリング



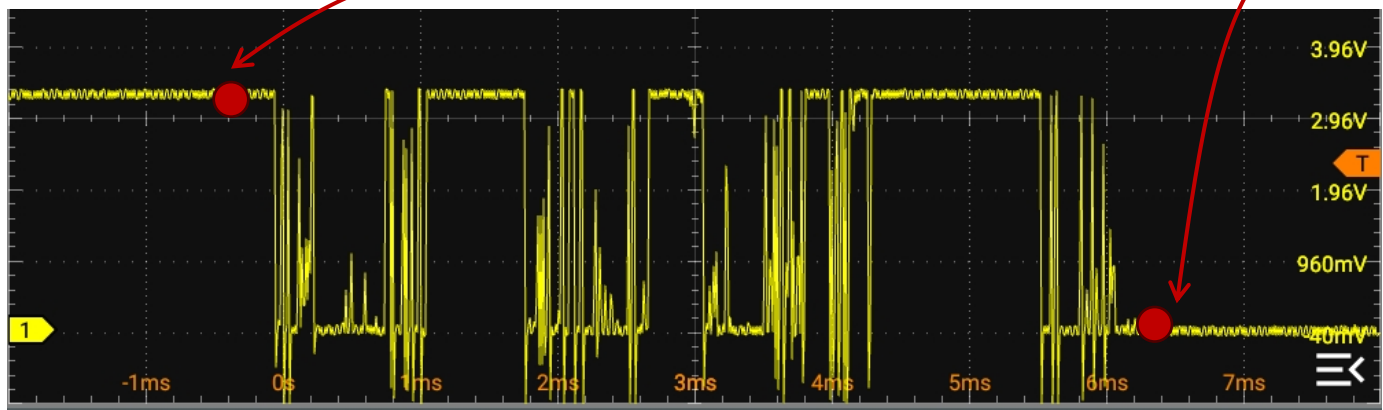
スイッチは論理が遷移するまでに10ms程度掛けて、論理が切り替わり続ける



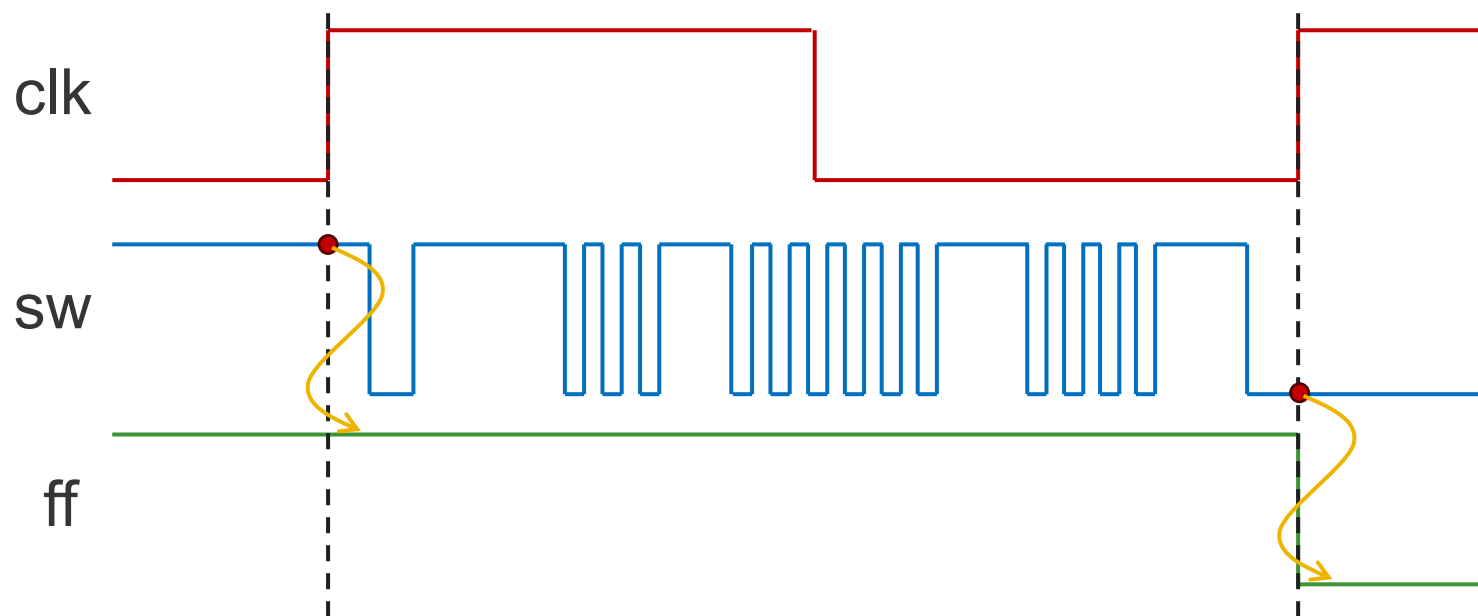
クロックとして用いると、1回の操作のはずなのに複数のステートが切り替わる原因となる。→ 誤動作しているように見える

安定した状態のみが入力されるようにしないとならない

# チャタリング除去の考え方



この2つのところでスイッチの状態をレジスタに取り込む



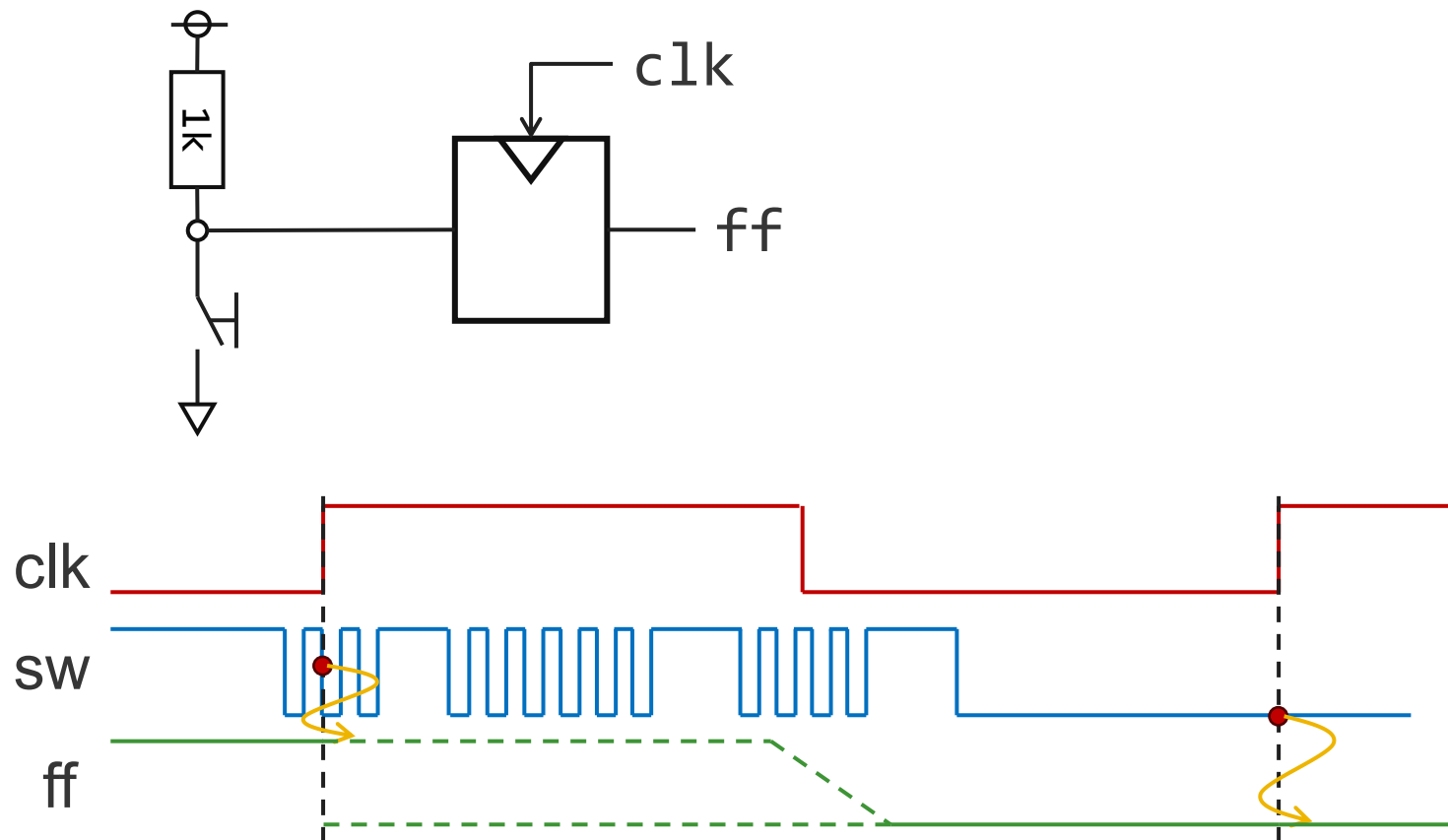
チャタリングより周期の長いクロックでスイッチの値を読み取る



チャタリングの影響を除いた信号を取得できる

# メタステーブル

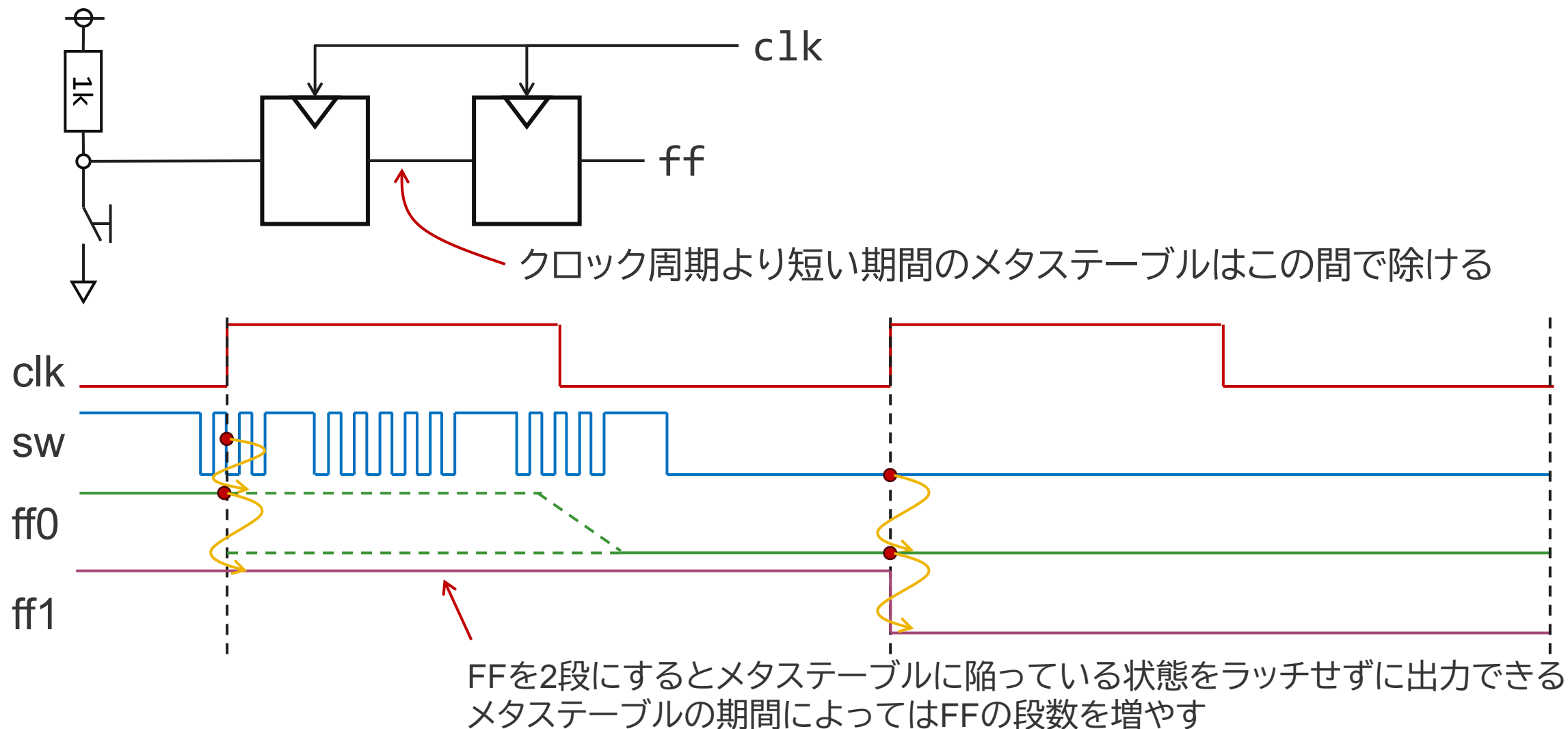
クロックとスイッチ入力是非同期なので、必ずしも安定した状態をラッチできるとは限らない



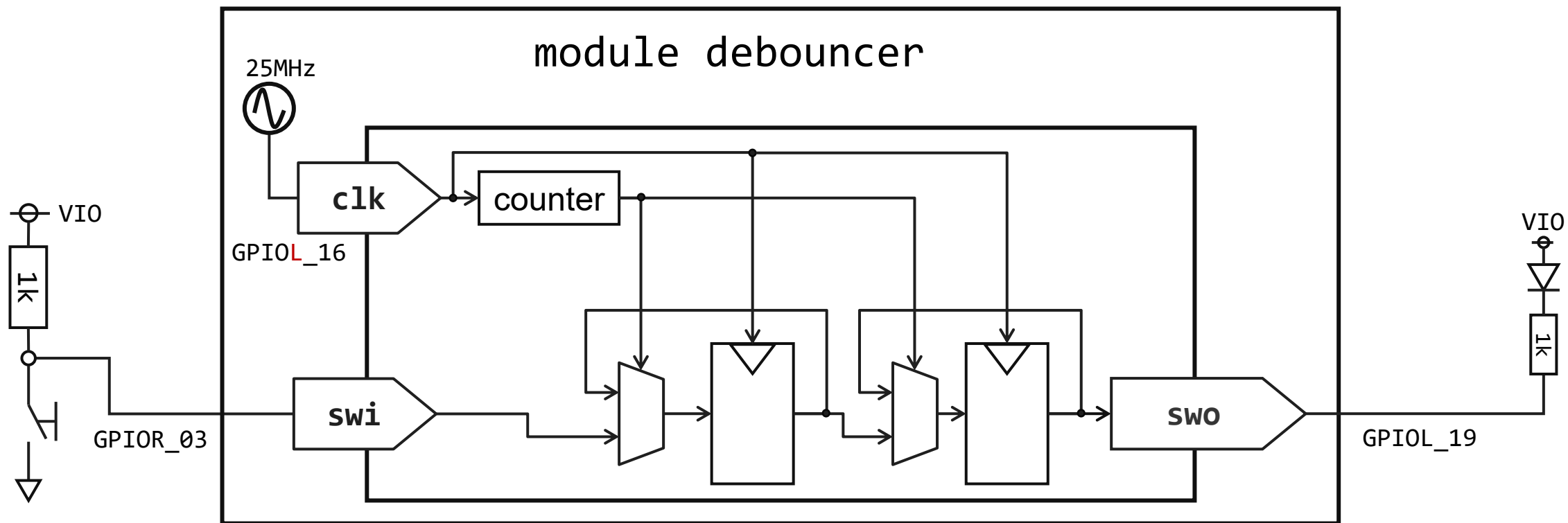
不安定な状態をラッチすると、出力が収束せずにフラフラする  
→ メタステーブルという

# メタステーブル除去

スイッチのときと同様にメタステーブルを除去するためのFFを追加する



# チャタリング除去の確認回路



# チャタリング除去のコード

※Top Module/Entitiyをdebouncerに変更するのを忘れずに

```
1 module debouncer(  
2   input wire swi,  
3   input wire clk,  
4   output reg swo);  
5  
6   localparam COUNT_MAX = 18'd250_000;  
7   reg [17:0] counter;  
8  
9   reg bounce;  
10  
11  
12   always @(posedge clk)  
13   begin  
14       if (counter <= COUNT_MAX)  
15           counter <= counter + 18'd1;  
16       else  
17           counter <= 18'd0;  
18   end  
19  
20   wire sw_flag;  
21   assign sw_flag = (counter == COUNT_MAX)? 1'b1 : 1'b0;  
22  
23   always @(posedge clk)  
24   begin  
25       if (sw_flag == 1'b1)  
26       begin  
27           bounce <= swi;  
28           swo <= bounce;  
29       end  
30   end  
31  
32 endmodule
```

25MHzから10ms(100Hz)を作るために25万回の  
カウント値を指定

10msのカウンター

10msに1回フラグを発生させる

フラグを用いて、レジスタへの書き込みを制御

レジスタ間の転送を記述



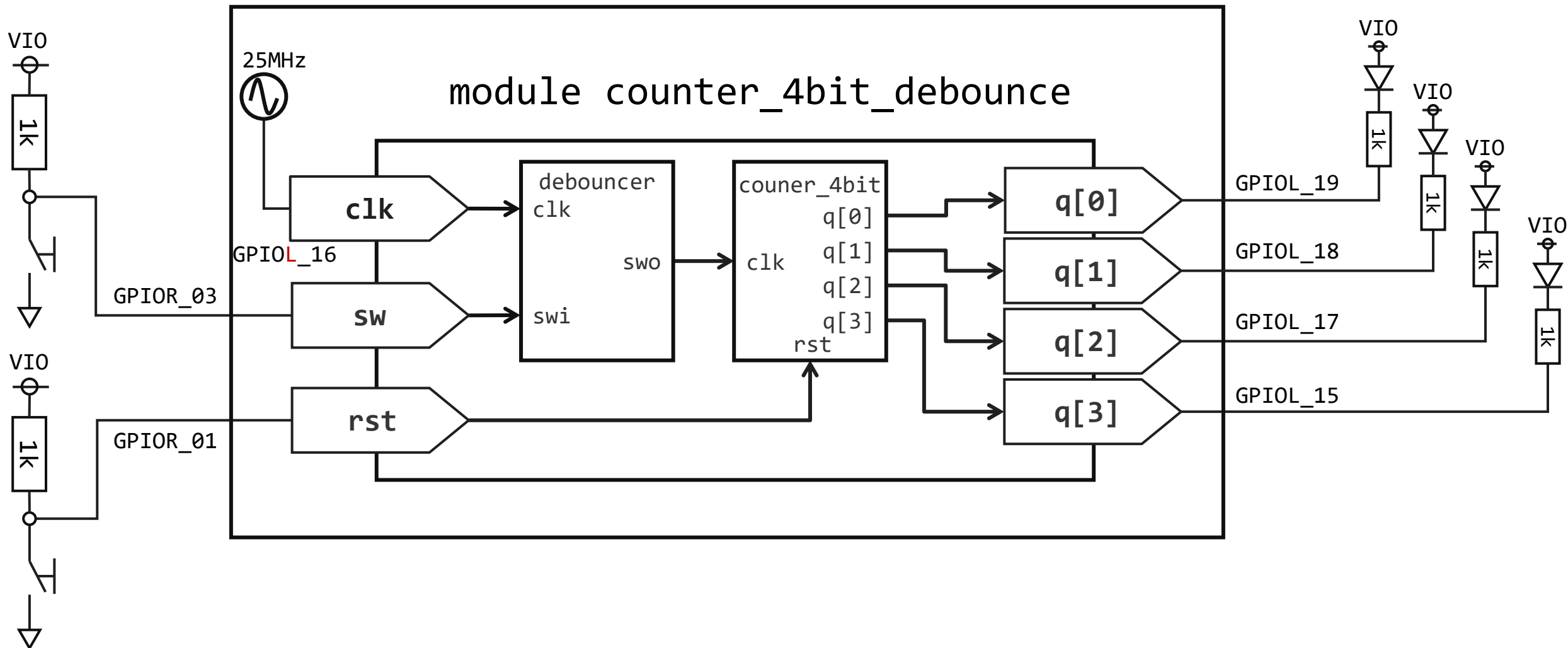
# ピンアサイン

GPIO : Instance View

|            |             |          | all      | all      | all      | all          |                |
|------------|-------------|----------|----------|----------|----------|--------------|----------------|
| Instance ^ | Package Pin | Resource | I/O Bank | Alt Conn | Features | Clock Region | Pad            |
| clk        | C2          | GPIOL_16 | 1B       | GCLK     | None     | L1           | GPIOL_16_CLK2  |
| swi        | A6          | GPIOR_03 | 2A       | None     | None     | R1           | GPIOR_03       |
| swo        | D3          | GPIOL_19 | 1B       | GCTRL    | None     | L1           | GPIOL_19_CTRL3 |

# チャタリング除去の効果の確認

4ビットカウンターとチャタリング除去を組み合わせる



# 4ビットカウンターとチャタリング除去の組み合わせ

```
1 module counter_4bit_debounce(  
2   input wire clk,   
3   input wire sw,   
4   input wire xrst,   
5   output wire [3:0] q);  
6  
7   wire cntclk;   
8  
9   debouncer sw_clk(  
10    .swi(sw),  
11    .clk(clk),  
12    .swo(cntclk));  
13  
14   counter_4bit counter(  
15    .clk(cntclk),  
16    .xrst(xrst),  
17    .q(q));  
18  
19   endmodule  
20
```

チャタリング除去のクロック入力

カウンタのクロックになるスイッチ入力

リセットのスイッチ入力

出力のLED

チャタリングが除去された信号のとなるwire

チャタリング除去モジュールのインスタンス

4ビットカウンタのインスタンス

# ピンアサイン

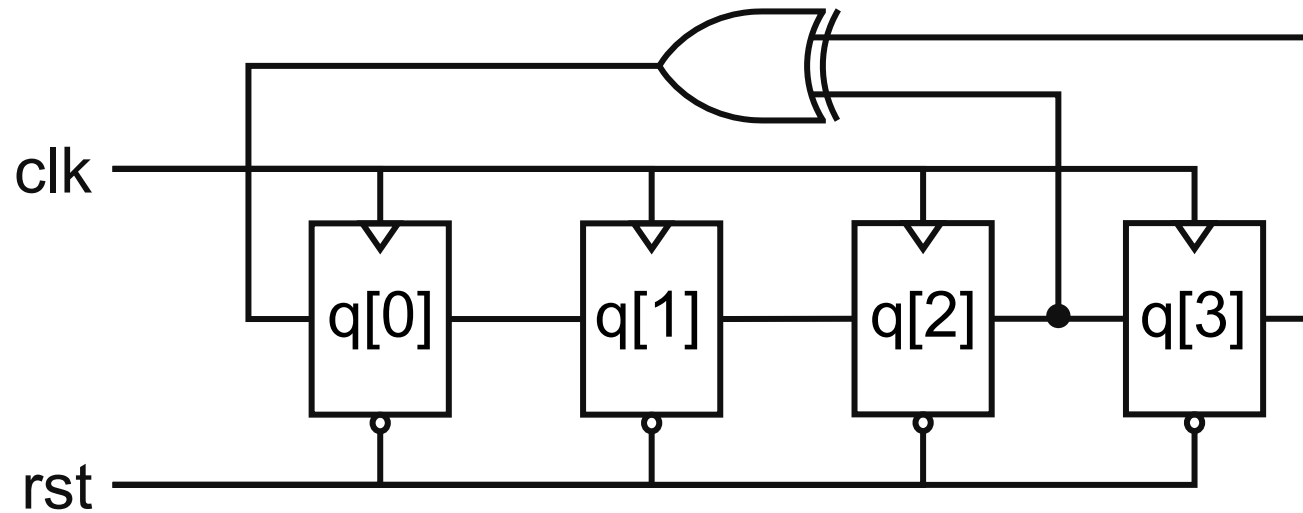
52

GPIO : Instance View

|            |             |          | all      | all      | all      | all          |                |
|------------|-------------|----------|----------|----------|----------|--------------|----------------|
| Instance ^ | Package Pin | Resource | I/O Bank | Alt Conn | Features | Clock Region | Pad            |
| clk        | C2          | GPIOL_16 | 1B       | GCLK     | None     | L1           | GPIOL_16_CLK2  |
| q[0]       | D3          | GPIOL_19 | 1B       | GCTRL    | None     | L1           | GPIOL_19_CTRL3 |
| q[1]       | E3          | GPIOL_18 | 1B       | GCTRL    | None     | L1           | GPIOL_18_CTRL2 |
| q[2]       | D2          | GPIOL_17 | 1B       | GCLK     | None     | L1           | GPIOL_17_CLK3  |
| q[3]       | E1          | GPIOL_15 | 1A       | GCLK     | None     | L0           | GPIOL_15_CLK1  |
| sw         | A6          | GPIOR_03 | 2A       | None     | None     | R1           | GPIOR_03       |
| xrst       | B5          | GPIOR_01 | 2A       | None     | None     | R1           | GPIOR_01       |

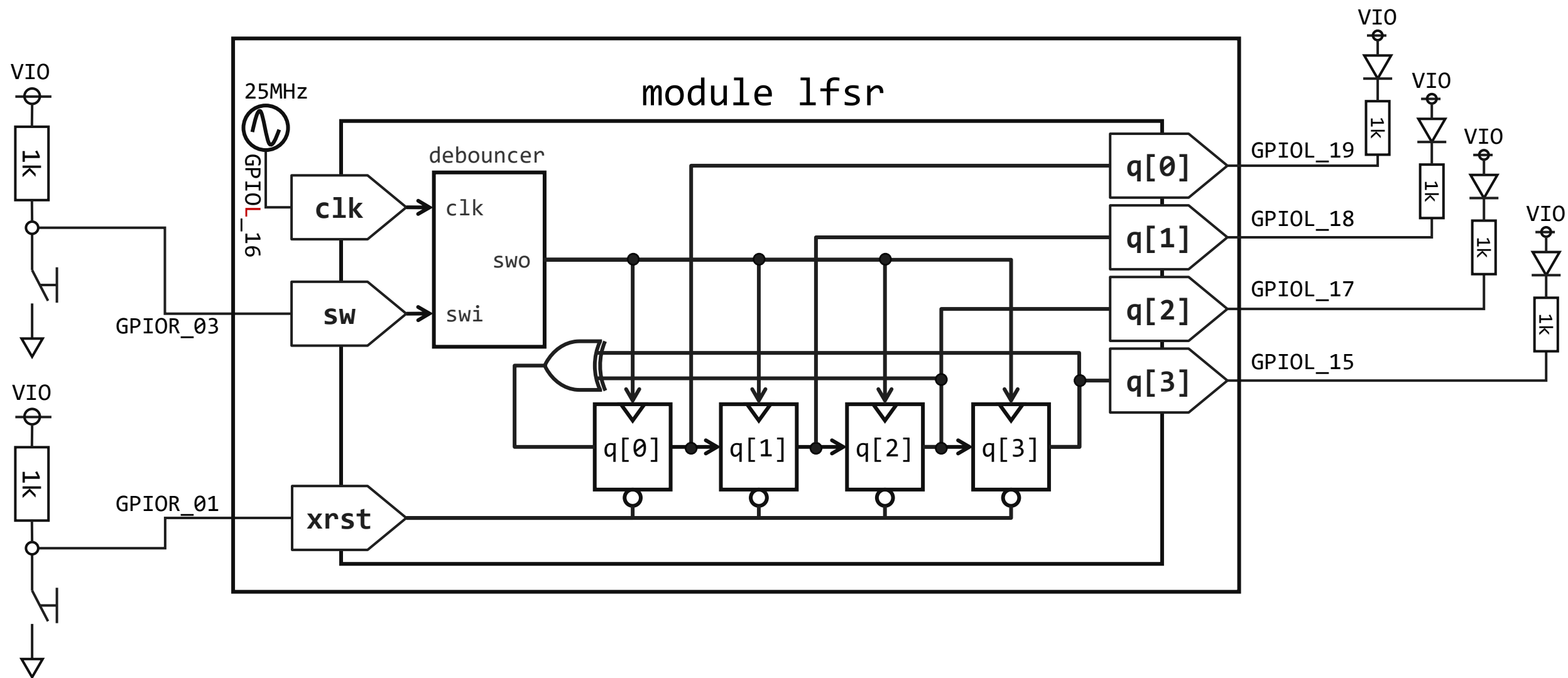
# 線形帰還シフトレジスタ

# 線形帰還シフトレジスタ



- Dフリップフロップを直列に繋いだ回路をシフトレジスタという
- シフトレジスタの途中(タップ)から信号を引き出し、排他的論理和を取ってからシフトレジスタに入力する回路を線形帰還シフトレジスタという。
- 疑似ランダム信号を出力する回路である。
- タップ位置によって周期が変わり、 $2^n - 1$ が最長周期となる。ただし、 $n$ はレジスタの個数。最長周期となるシーケンスはM系列と呼ばれる。
- リセットは**全ビットが0**以外となるようにする。

# 線形帰還シフトレジスタの検証回路



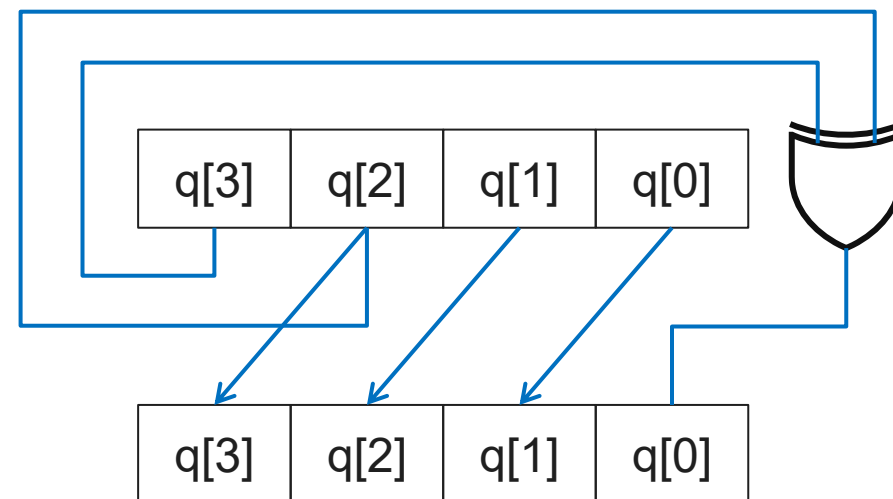
# Verilog コード

56

```
lfsr.v debouncer.v counter_4bit_debounce.v
1 module lfsr(
2   input wire clk,
3   input wire sw,
4   input wire xrst,
5   output reg [3:0] q);
6
7   wire cntclk;
8
9   debouncer sw_clk(
10    .swi(sw),
11    .clk(clk),
12    .swo(cntclk));
13
14   always @(posedge cntclk)
15   begin
16     if (!xrst)
17     begin
18       q <= 4'b1;
19     end
20     else
21     begin
22       q <= {q[2:0], q[3]^q[2]};
23     end
24   end
25
26 endmodule
```

0でリセットすると、動かなくなるため1でリセット

LFSR処理





# ピンアサイン

57

GPIO : Instance View

|            |             |          | all      | all      | all      | all          |                |
|------------|-------------|----------|----------|----------|----------|--------------|----------------|
| Instance ^ | Package Pin | Resource | I/O Bank | Alt Conn | Features | Clock Region | Pad            |
| clk        | C2          | GPIOL_16 | 1B       | GCLK     | None     | L1           | GPIOL_16_CLK2  |
| q[0]       | D3          | GPIOL_19 | 1B       | GCTRL    | None     | L1           | GPIOL_19_CTRL3 |
| q[1]       | E3          | GPIOL_18 | 1B       | GCTRL    | None     | L1           | GPIOL_18_CTRL2 |
| q[2]       | D2          | GPIOL_17 | 1B       | GCLK     | None     | L1           | GPIOL_17_CLK3  |
| q[3]       | E1          | GPIOL_15 | 1A       | GCLK     | None     | L0           | GPIOL_15_CLK1  |
| sw         | A6          | GPIOR_03 | 2A       | None     | None     | R1           | GPIOR_03       |
| xrst       | B5          | GPIOR_01 | 2A       | None     | None     | R1           | GPIOR_01       |

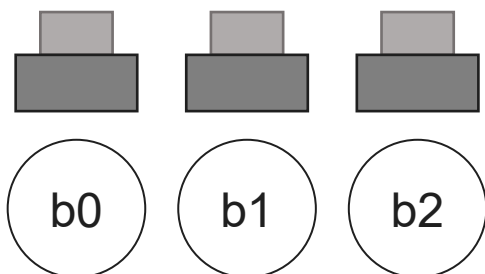
# パスワード認証回路

ステートマシンの学習

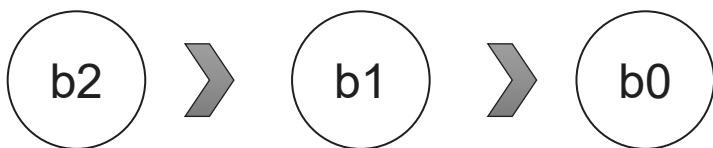
# パスワードをチェックする

3つのボタンが正しい順番に押されたときのみ解錠の信号を出力するステートマシンを設計する

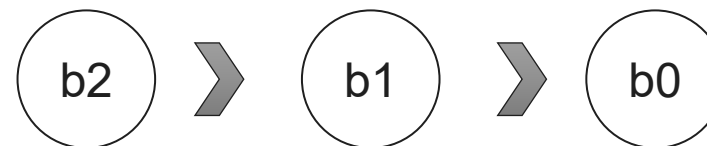
3つの解錠用暗証コード入力ボタン



正しい解錠コードの入力順番

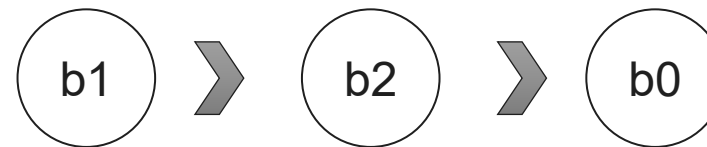


正しい解錠コードを入力したとき



LED    

誤ったコードを入力したとき



LED    

# ステートマシン設計

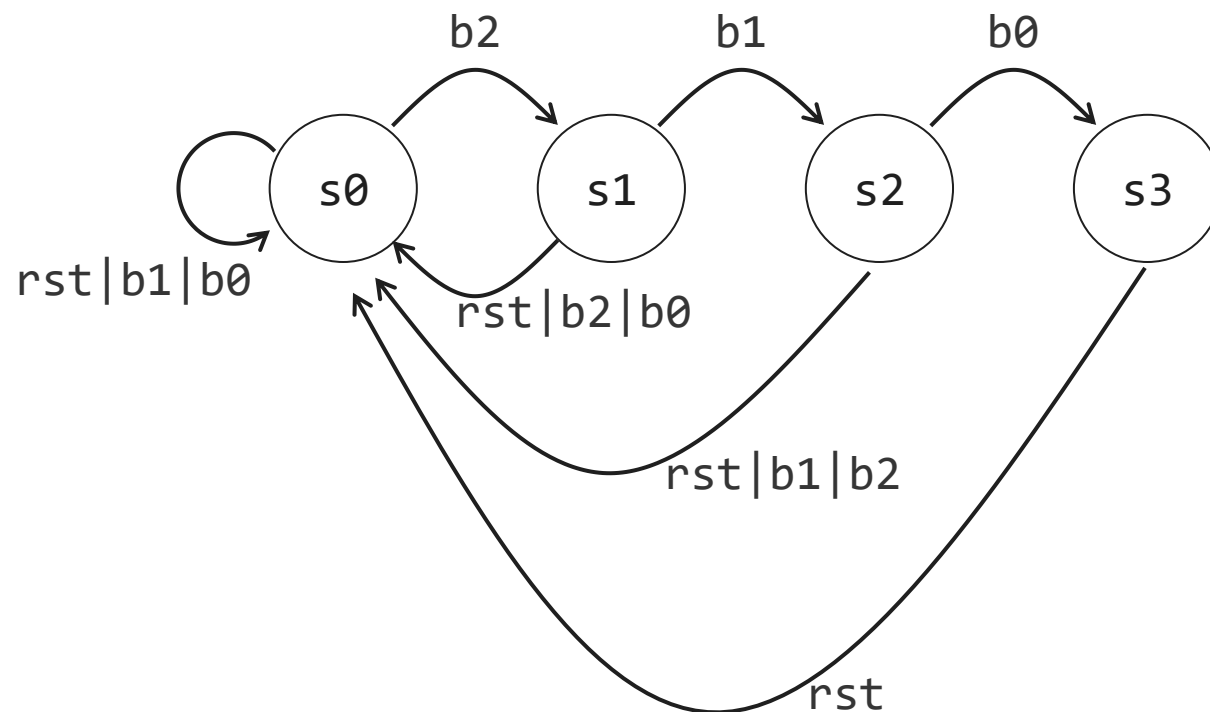
動作から内部状態の個数を決める

1. 待機状態.....s0
2. 2個目の待機.....s1
3. 3個目の待機.....s2
4. 成功表示.....s3

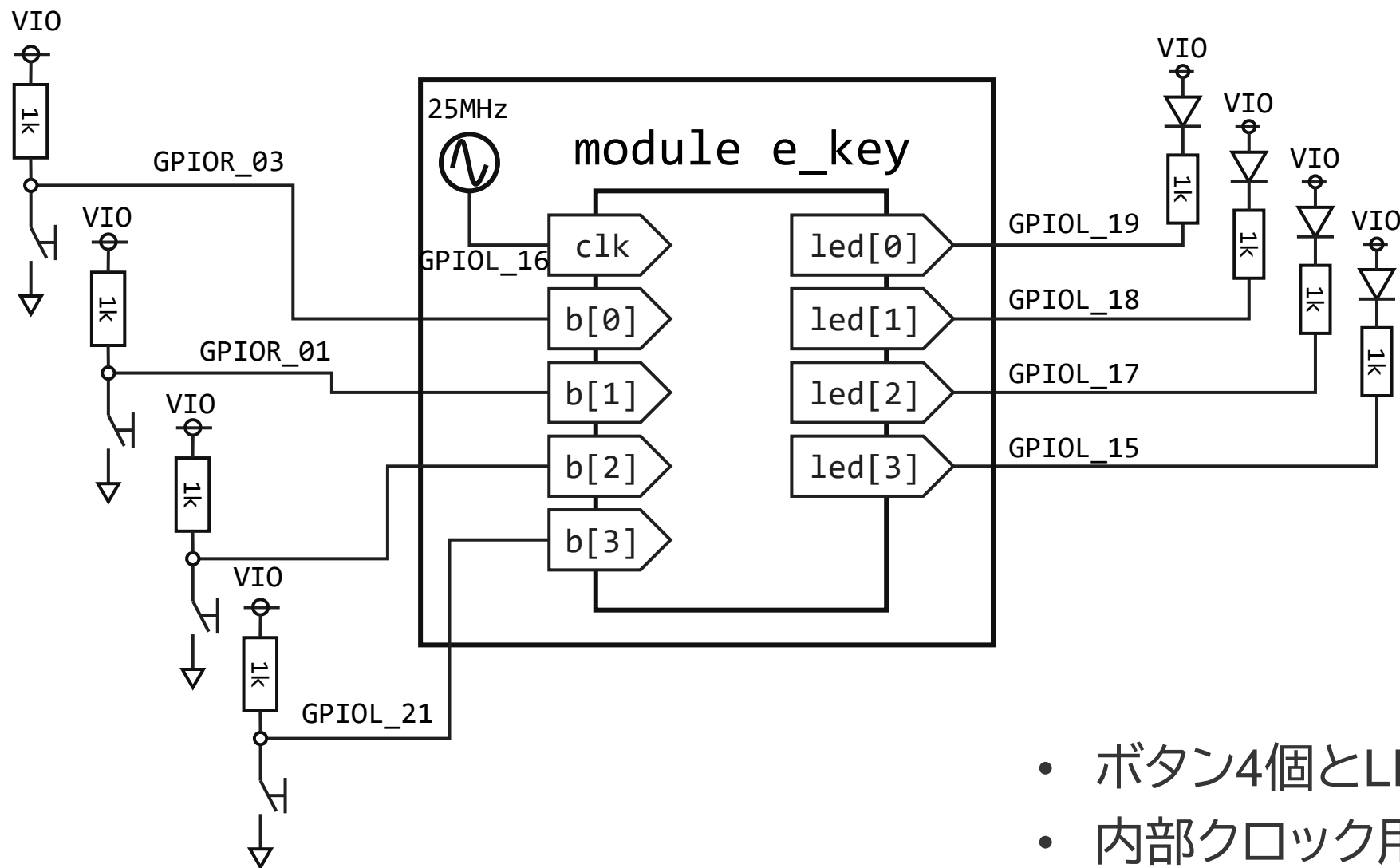
LEDの表示

| state | led     |
|-------|---------|
| s0    | 4'b1111 |
| s1    | 4'b0111 |
| s2    | 4'b0011 |
| s3    | 4'b0110 |

ステートマシン



# ハードウェア設計



- ボタン4個とLED4個を持つ
- 内部クロック用の25MHz信号を入力

# Verilogコード1

## スイッチ処理

```

1 module e_key(
2   input wire clk,
3   input wire [2:0] b,
4   input wire xrst,
5   output reg [3:0] led);
6
7   localparam S0 = 2'b00;
8   localparam S1 = 2'b01;
9   localparam S2 = 2'b10;
10  localparam S3 = 2'b11;
11

```

localparamを用いる  
とモジュール内で使える  
定数を定義できる

```

12 wire [2:0] b_debounce;
13 wire [2:0] b_edge;
14 reg [2:0] b_prev;
15
16 debouncer b0
17 (
18   .swi(b[0]),
19   .clk(clk),
20   .swo(b_debounce[0])
21 );
22
23 debouncer b1
24 (
25   .swi(b[1]),
26   .clk(clk),
27   .swo(b_debounce[1])
28 );
29
30 debouncer b2
31 (
32   .swi(b[2]),
33   .clk(clk),
34   .swo(b_debounce[2])
35 );
36
37 always @(posedge clk)
38 begin
39   b_prev <= b_debounce;
40 end
41
42 assign b_edge = b_prev & (~b_debounce);
43

```

チャタリング除去

エッジ検出  
1つ前の値と現在の値を比較して立ち下がりを検出

# Verilogコード2

44 `reg [1:0] next_state, current_state;` ← ステートを保持するレジスタ定義

46 `always @(posedge clk)`

47 `begin`

48 `if (!xrst) current_state <= S0;`

49 `else current_state <= next_state;`

50 `end`

52 `always @(*)`

53 `begin`

54 `if (!xrst)`

55 `begin`

56 `next_state = S0;`

57 `end`

58 `else`

59 `begin`

60 `case(current_state)`

61 `S0:begin`

62 `if (b_edge == 3'b000) next_state = S0;`

63 `else if (b_edge == 3'b100) next_state = S1;`

64 `else next_state = S0;`

65 `end`

66 `S1:begin`

67 `if (b_edge == 3'b000) next_state = S1;`

68 `else if (b_edge == 3'b010) next_state = S2;`

69 `else next_state = S0;`

70 `end`

71 `S2: begin`

72 `if (b_edge == 3'b000) next_state = S2;`

73 `else if (b_edge == 3'b001) next_state = S3;`

74 `else next_state = S0;`

75 `end`

76 `S3: begin`

77 `if (!xrst) next_state = S0;`

78 `else next_state = S3;`

79 `end`

80 `default: next_state = S0;`

81 `endcase`

82 `end`

83 `end`

ステートの更新を記述

ムーアマシン

入力 →

記憶回路

組み合わせ回路

→ 出力

組み合わせ回路

current\_state  
→ レジスタ出力

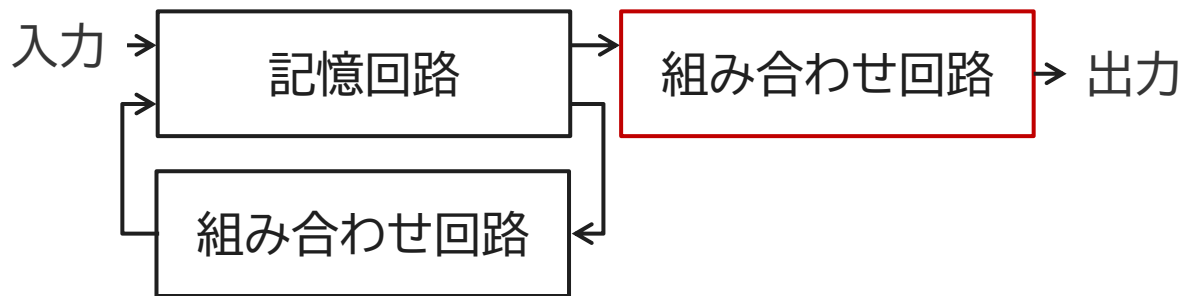
next\_state  
→ 組み合わせ回路

状態更新の組み合わせ回路

# Verilogコード3

## led出力の組み合わせ回路+レジスタ代入

```
85 always @(posedge clk)
86 begin
87     if (!xrst) led <= 4'b1111;
88     else
89     begin
90         case(current_state)
91             S0: led <= 4'b1111;
92             S1: led <= 4'b0111;
93             S2: led <= 4'b0011;
94             S3: led <= 4'b0110;
95             default: led <= 4'b1111;
96         endcase
97     end
98 end
99
100 endmodule
101
```



現在の状態を元にled出力信号を生成

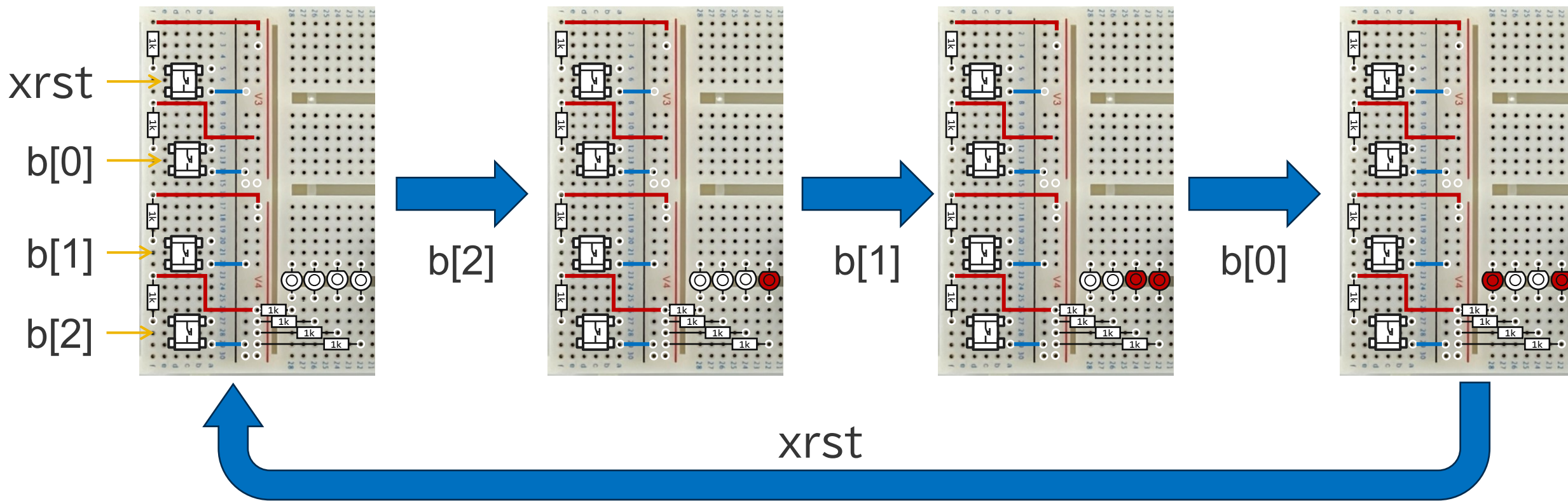


GPIO : Instance View

| Instance | Package Pin | Resource | I/O Bank | Alt Conn | Features | Clock Region | Pad              |
|----------|-------------|----------|----------|----------|----------|--------------|------------------|
| b[0]     | B5          | GPIOR_01 | 2A       | None     | None     | R1           | GPIOR_01         |
| b[1]     | A5          | GPIOR_00 | 2A       | None     | None     | R1           | GPIOR_00         |
| b[2]     | B3          | GPIOL_21 | 1B       | None     | None     | L1           | GPIOL_21_NSTATUS |
| clk      | C2          | GPIOL_16 | 1B       | GCLK     | None     | L1           | GPIOL_16_CLK2    |
| led[0]   | D3          | GPIOL_19 | 1B       | GCTRL    | None     | L1           | GPIOL_19_CTRL3   |
| led[1]   | E3          | GPIOL_18 | 1B       | GCTRL    | None     | L1           | GPIOL_18_CTRL2   |
| led[2]   | D2          | GPIOL_17 | 1B       | GCLK     | None     | L1           | GPIOL_17_CLK3    |
| led[3]   | E1          | GPIOL_15 | 1A       | GCLK     | None     | L0           | GPIOL_15_CLK1    |
| xrst     | A6          | GPIOR_03 | 2A       | None     | None     | R1           | GPIOR_03         |

# 動作

リセット直後

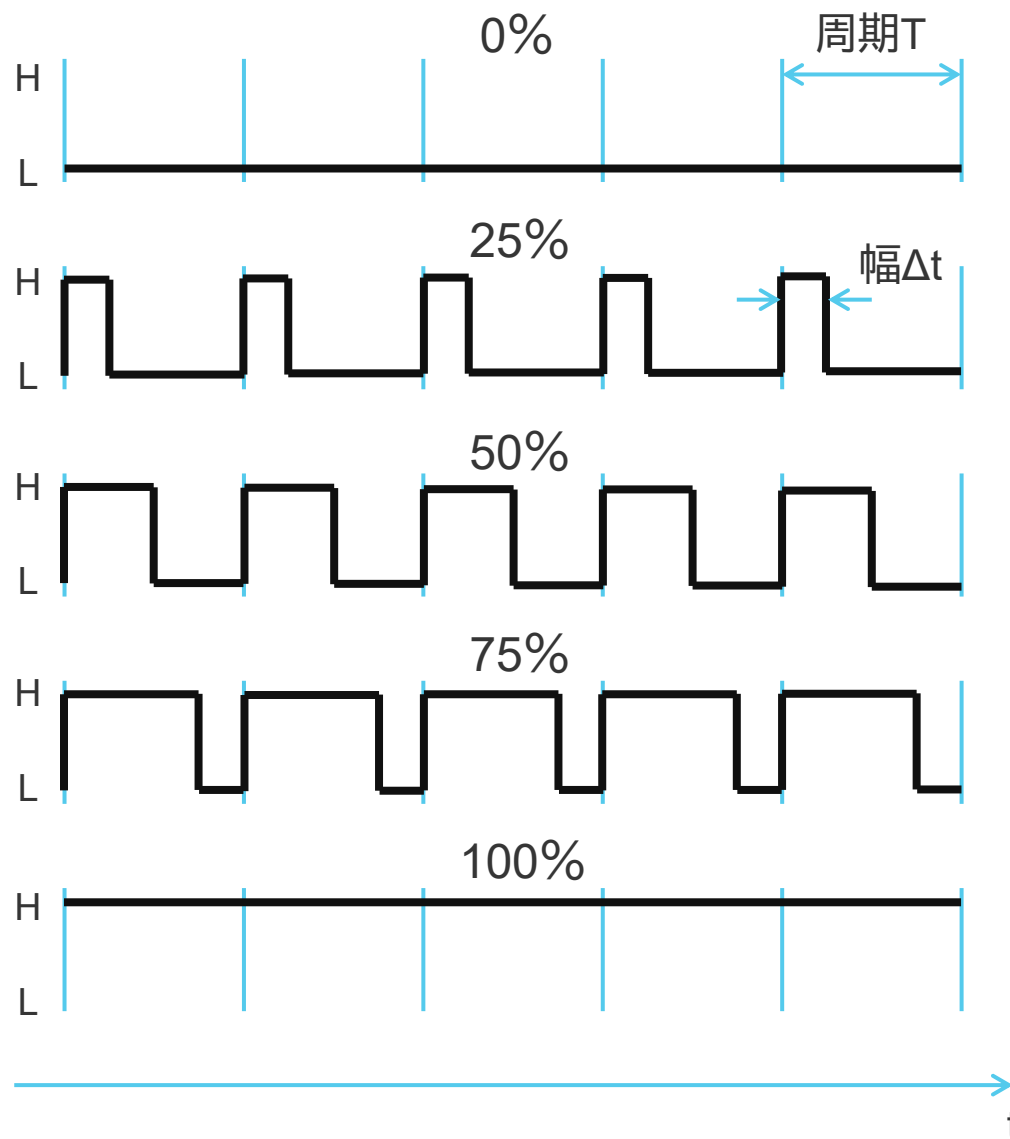


順番を間違えるとリセット直後に戻ること確かめよう

# 今日の発展課題

# PWMを用いたLEDの調光

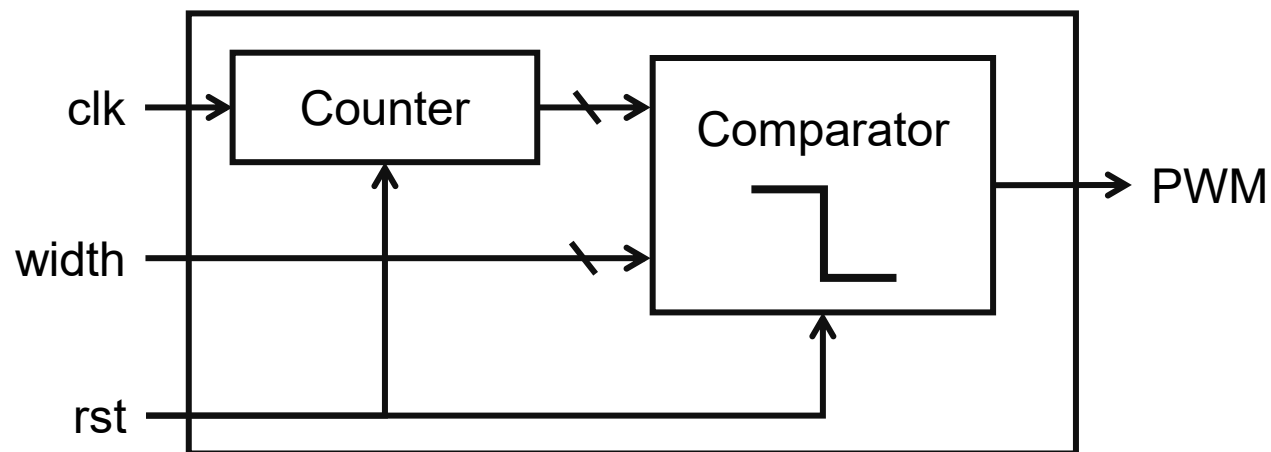
# PWM (Pulse Width Modulation)とは？



- パルスの幅 $\Delta t$ を変調することで情報を表現する方法
- 周期 $T$ は一定
- 波形を平均することでアナログ電圧が得られる
- **PWM波形でLEDを点灯することで、LEDを調光できる**

# PWMの作り方

## PWMモジュール



- PWMを作るにはカウンタと比較器を組み合わせる。
- カウンタはカウントアップしていて、最大値に到達したら初期値に戻る。
- カウンタの値と目標値を比較して、超えていたら出力をLに、超えていなければ出力をHにする。

